



Data Pipeline & Data Observability

AICB-P1 · Ngày 10 · Khi pipeline hỏng, agent không báo lỗi — nó trả lời sai một cách tự tin

Tên Giảng Viên

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

“Pipeline nạp tài liệu cho agent hỏng lúc 2 giờ sáng. Không có alert nào. Agent vẫn trả lời — tự tin, trôi chảy, và sai. Bạn biết lúc nào?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Từ Internet Boom Đến Agent

2. Data Pipeline Fundamentals

3. Ingestion — Thu Thập Data Từ Nhiều Nguồn

4. Transform — Làm Sạch & Chuẩn Hóa Data

5. Storage & Serving Cho Agent

6. Data Quality — 6 Dimensions

7. Quality Gates & Data Contracts

8. Từ Data-issue Đến Agent-symptom

9. Data Observability

10. Phát Hiện Bất Thường

11. Lineage, Governance & Tuân Thủ

12. Alerting, SLA & Incident Runbook

13. Orchestration & Automation

14. Chi Phí & Kinh Tế Pipeline

15. Sự Cố Có Thật & Những Con Số Đùng Dùng

16. Frontier 2026

17. Lab 10, Checklist & Tổng Kết

18. Phụ Lục A — Cầu Nối Sang Ngày 13

- Một agent RAG xuất sắc vẫn **trả lời sai** nếu vector store được nạp data bẩn hoặc data cũ
- **Garbage in** → **garbage out**: chất lượng output bị chặn trên bởi chất lượng input data
- Data work bị **đánh giá thấp một cách hệ thống** — Sambasivan et al. (Google Research, CHI 2021) gọi đây là *data cascades*: lỗi data ở thượng nguồn nhân lên thành hồng hóc ở hạ nguồn
- Observability = cơ chế phát hiện data sai **trước khi user phàn nàn**

Con số bạn sẽ nghe rất nhiều

“80% thời gian data science là làm sạch dữ liệu”

Con số này **không đáng tin**. Ta sẽ mở xẻ nó ở **\$15** — và dùng nó để học cách kiểm chứng một trích dẫn.

BI Pipeline (batch, dashboard)

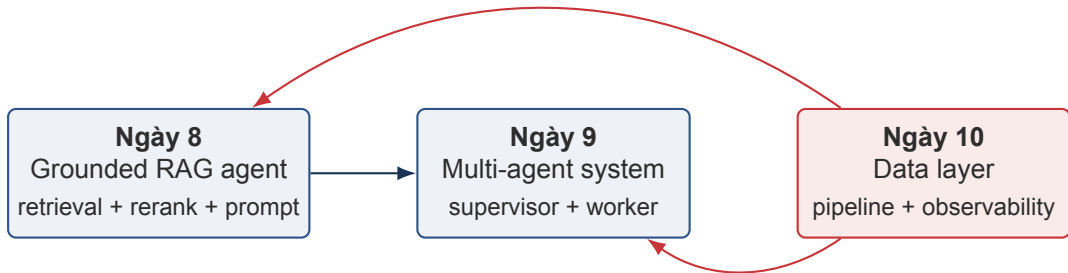
- **Hợp đồng đúng-dẫn:** con số trên dashboard khớp con số ở hệ nguồn tính đến lần refresh gần nhất
- Batch-oriented — chịu được **stale vài giờ** là bình thường
- Khi hỏng: **fail LOUDLY** — dashboard vỡ, và **bất kỳ ai nhìn dashboard** đều thấy

RAG / Agent Corpus (retrieval)

- **Hợp đồng đúng-dẫn:** chunk được retrieve vừa **liên quan chủ đề** vừa **không stale / không bị mâu thuẫn** bởi bản mới hơn
- Không có ngưỡng “stale bao lâu thì được” rõ ràng như BI
- Khi hỏng: **fail SILENTLY** — không exception nào được ném ra, và **không ai** biết cho đến khi user complain

Cùng một phân biệt xuất hiện độc lập trong benchmark của dbt Labs (2026-04): semantic-layer fail thì query báo lỗi; raw text-to-SQL fail thì ra một con số trông hợp lý (dbt Labs, 2026-04).

Sự bất đối xứng này là lý do Ngày 10 tồn tại.



- Ngày 8: bạn dựng một agent **trả lời đúng nhờ retrieval** — nhưng retrieval chỉ đúng nếu vector store được nạp data sạch, mới
- Ngày 9: bạn dựng **nhiều agent phối hợp** — một worker đọc data hổng thì lỗi lan sang toàn bộ hệ thống multi-agent
- Ngày 10: **cả hai đều đứng trên cùng một tầng nền** — pipeline nạp data và cơ chế phát hiện khi tầng nền đó hổng

Từ Internet Boom Đến Agent

Data đã thay đổi thế nào — và vì sao câu hỏi cuối cùng là “còn tin được không”

- Phần này **không** kể một câu chuyện bốn giai đoạn nổi tiệp. Nó kể **bốn câu hỏi chồng lẫn nhau**, không bàn giao gọn gàng
- Note “3 Vs” của Doug Laney (2001) đi trước Hadoop (2006) **5 năm**: “lưu theo hình gì” (BI/marketing) và “xử lý ở quy mô nào” (distributed systems) được hỏi **cùng lúc**, bởi hai nhóm khác nhau
- Mỗi era **cộng thêm** vào era trước, không thay thế: hệ thống 2026 vẫn phải store, process ở quy mô petabyte, learn từ data có nhãn — và giờ còn phải được một **agent** tin
- “Tin được không” **không phải phát minh của 2020**: data-centric AI (Andrew Ng, IEEE Spectrum, 2022-02-09) đã hỏi nó trước, từ bài toán chất lượng **nhãn** trong supervised learning

Bốn câu hỏi, nguyên văn

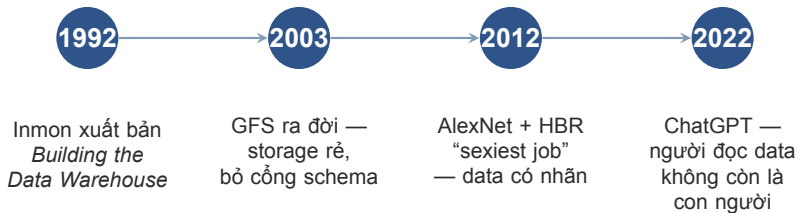
1992–2003: Lưu theo hình dạng nào?

2003–2014: Ai còn kiểm tra ở cửa vào?

2007–2015: Học được từ nó không?

2016–2020: Người còn tin được nó không?

2020–2026: Còn tin được — khi người đọc không còn biết nghi ngờ?



- **Data trông như thế nào:** có cấu trúc, dạng bảng, gần tuyệt đối — web LAMP (PHP, công bố 8/6/1995; MySQL, bản nội bộ đầu tiên 23/5/1995) sinh thêm *dòng* (đơn hàng, session), chưa sinh document hay embedding
- Batch là **mặc định và kỳ vọng**: warehouse “mới đến hôm qua” không bị coi là vấn đề
- Mốc quy mô có tên, có ngày: Teradata dựng warehouse **>1 TB** đầu tiên cho Walmart, 1992 — không suy ra đây là DB lớn nhất era
- “**Hỏng**” nghĩa là gì: job ETL đêm fail, join key lệch, schema lệch giữa nguồn và warehouse — **chưa ai hỏi về nghĩa hay độ tươi** của nội dung
- **Ai tiêu thụ:** một nhóm nhỏ, chuyên biệt — BI analyst và lãnh đạo, qua report định kỳ

Inmon (1992, *Building the DW*) vs. Kimball (1996, **4 năm sau**) — không cãi nhau về *có nên lưu*, mà về *lưu theo hình gì*: normalized-trước hay star-schema-trước.

Doug Laney, 2001 — không phải Gartner

Tại **META Group**, không phải Gartner: “3D Data Management” (2/2001) đặt ra **3 Vs.** Gartner chỉ **đồng ý mua META** (12/2004, \$162tr) rồi thừa kế credit. “Big data” **không có** trong note gốc (Forbes, 2013-05-09)

“...the typical piece of information will never be looked at by a human being.” — Michael Lesk, 1997

Schema-on-Write (di sản Era 1)

- Warehouse trả giá schema **trước khi ghi** — bản ghi xấu bị từ chối ngay tại cửa ingest
- Ổn với GB dòng giao dịch có cấu trúc; sụp đổ trước log, clickstream, trang web crawl sinh nhanh và đa hình hơn bất kỳ ai kịp model trước
- Đây là nút thắt mà **GFS** (10/2003) và **MapReduce** (12/2004), hai paper của Google, tháo ra

Schema-on-Read (Hadoop, NoSQL, data lake)

GFS: “...failures are the norm rather than the exception” — ghi **append** (Ghemawat et al., SOSP '03). **Bigtable** (2006) & **Dynamo** (2007): hai paper **STORAGE** cùng cửa sổ — nhấn “chỉ process” bỏ rơi cả hai.

“...various users of the lake can come to examine, dive in, or take samples.” — James Dixon, 2010-10-14

Cái giá thầm lặng: storage rẻ + schema-on-read = **không ai validate ở cửa vào nữa**.

2007–2015 — Học được từ nó không?

- **Corpus có nhãn:** ImageNet, CVPR 2009 (6 đồng tác giả, đều Princeton) — công bố **3,2 triệu ảnh/5.247 synset**; bộ đầy đủ 14,2 triệu ảnh đến **sau**
- **AlexNet:** kết quả công bố **12/10/2012**, workshop ILSVRC2012 (trong ECCV 2012). Top-5 error **15,3%** so **26,2%** hạng nhì. Đăng tại **NIPS 2012** — “NeurIPS” chỉ có từ 2018
- **Không** phải CNN đầu tiên (LeNet, 1989/1998) — cái mới 2012 là **đủ data có nhãn + đủ GPU**
- “Sexiest Job of the 21st Century” (Davenport & Patil, HBR 10/2012) — bản 2022 cùng tác giả: nhu cầu **tăng**, vai trò phân mảnh thành ML/data engineer

2016–2020 — Người còn tin được nó không?

ELT thay ETL vì **kinh tế** (Snowflake, GA 6/2015). **86% analyst dùng data lỗi thời, 62% chờ engineering** nhiều lần/tháng (khảo sát Dimensional Research/Fivetran, qua CIDR '21).

Format	Công ty	Ngày
Hudi	Uber	2017-03-11
Iceberg	Netflix	2018-11
Delta Lake	Databricks	2019-04-24

dbt (2016), Great Expectations (2017), data mesh (2019-05), “data observability” (Moses **tự nhận**, 2019) — từ vụng §7/§9/§11/§13, xây cho một người đang nhìn dashboard.

- ANN library rồi vector DB: FAISS mở nguồn 29/3/2017 → Weaviate/Milvus/Pinecone (2019) → Chroma (repo 5/10/2022, **8 tuần trước ChatGPT**)
- LangChain (17/10/2022) và LlamaIndex (2/11/2022) — **cả hai ra đời trước ChatGPT** (30/11/2022)
- **MCP** (Anthropic, 2024-11-25): chuẩn hoá cách agent chạm vào data — trước đó mỗi kết nối là một tích hợp riêng, không dựng nổi tầng governance nhất quán
- 2026: Fivetran+dbt Labs **hoàn tất sáp nhập** 1/6, tiêu đề thông cáo nhắc thẳng “**Trusted AI Agents**”; Prefect **công bố đồng ý** mua Dagster 13/7 (**chưa đóng**)

Document nội bộ phi cấu trúc thành **hạ tầng production** có SLA độ tươi. Embedding pipeline phải liên tục đồng bộ với corpus — đổi model **vô hiệu toàn bộ index cùng lúc**. “**Hồng**”: BI = timestamp cũ ai cũng thấy; RAG = **câu tự tin và sai, không dấu hiệu**.

“...the focus has to shift from big data to good data...” — Andrew Ng, “Unbiggen AI”, IEEE Spectrum, 2022-02-09

Từ 2017 đến 2019, data team đã dựng xong **gần như toàn bộ bộ đồ nghề của Ngày 10** — dbt test, Great Expectations, ACID table format, lineage, chính chữ “data observability” — **cho một con người đang nhìn dashboard.**

Cái Era 4 lấy đi là chính con người đó.

Trong BI, data cũ hiện ra thành **một timestamp cũ mà ai đó nhìn thấy**. Trong RAG, data cũ hiện ra thành **một câu văn tự tin và sai** — không một dấu hiệu nào báo là nó cũ.

“The vast majority of customers had less than a terabyte of data in total data storage.” / “90% of queries processed less than 100 MB of data.” — Jordan Tigani, “Big Data is Dead”, MotherDuck, ~2023-02 (bài không ghi ngày; suy từ thời điểm đăng trên Hacker News)

*Đây là mẫu truy vấn **con người** mà era agent phá vỡ: nhỏ, thiên về dữ liệu mới, dễ tha thứ. Bài toán volume vừa được giải xong (§14) đúng lúc một bài toán khó hơn chiếm chỗ: thứ đang đọc data này có biết lúc nào nó sai không?*

invented=bắt đầu tồn tại **published**=công bố công khai **open-sourced**=mã nguồn phát hành công khai
became-dominant=có bằng chứng đã thắng thị trường/benchmark. Trộn bốn loại này — hoặc với *announced/GA/governance* — là lỗi phổ biến nhất của slide lịch sử.

Ngay cả nghiên cứu viên dựng section này cũng mắc đúng lỗi này ba lần ở bản nháp đầu — rồi tự bắt và tự sửa:

Trường hợp	Bản nháp đầu (nhầm loại ngày)	Sự thật, sau khi tự sửa
MySQL	“Open-sourced năm 1995”	23/5/1995 là bản phát hành nội bộ đầu tiên. GPL đến nhiều năm sau — không có ngày nào xác minh được, đừng đoán
Apache Spark	“Trở thành ASF Top-Level Project (2/2014) = ngày open-source”	TLP là sự kiện quản trị . Spark đã mở nguồn BSD từ 2010 ; hiến cho ASF và đổi Apache 2.0 năm 2013
Redshift vs. Big-Query	“Redshift là cloud data warehouse đầu tiên”	BigQuery announced 5/2010, GA 2012 — đi trước cả bản limited preview 11/2012 của Redshift (GA 2013)

Nguồn đối chiếu: Wikipedia (MySQL, Apache Spark, BigQuery), Databricks blog (2014-11-05), AWS (2012-11-28).

Lưu ý: “90% data thế giới được tạo trong 2 năm qua” — SINTEF/ScienceDaily, 2013-05-22, **không một citation, không phương pháp nào**. Lặp mỗi năm với cùng cửa sổ “hai năm qua” ⇒ **không thể phản nghiệm**: chưa ai tổng điều tra “data của thế giới”.

Lưu ý: “Gartner đặt ra 3 Vs năm 2001” — sai người, đúng năm: tác giả là Doug Laney tại **META Group**; Gartner chỉ đồng ý mua lại công ty đó 3 năm sau rồi thừa kế credit.

Lưu ý: “80–90% data doanh nghiệp là unstructured” — truy về Shilakes & Tylman, Merrill Lynch, 1998-11-16. Wikipedia: “*it is unclear what the source of this number is.*” Debunk có tên: Seth Grimes, 2008-08-01.

Ngay cả bảng sửa sai attribution cũng có thể chứa sai attribution mới

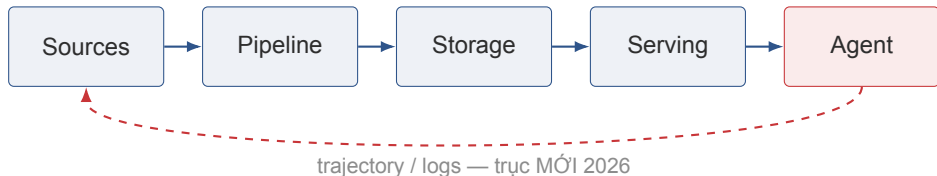
Câu “Data is the new oil” được gán cho Clive Humby (2006) — nhưng **nửa sau nổi tiếng** (“valuable, but if unrefined it cannot really be used”) là lời của **Michael Palmer**, người diễn giải Humby nhiều tháng sau, không phải Humby nói. §15: phương pháp kiểm chứng & các sự cố có thật.

Data Pipeline Fundamentals

Từ nguồn đến agent — ETL, ELT, Batch, Streaming

02

Data pipeline — Chuỗi bước tự động di chuyển dữ liệu từ hệ thống nguồn đến nơi được dùng — gồm **extract** (lấy), **transform** (biến đổi), **load** (nạp), dù thứ tự ba bước này thay đổi tùy kiến trúc (xem ETL vs ELT, trang sau).



Chiều ngược (đỏ, nét đứt) là điều 2026 thêm vào sơ đồ cũ: chính agent sinh ra dữ liệu quay lại nuôi pipeline — chi tiết trang sau.

- Mỗi tool call, mỗi bước reasoning trung gian, mỗi lượt retrieval, mỗi lượt handoff giữa các agent (Ngày 9) đều tạo ra dữ liệu có cấu trúc mới
- Dữ liệu này **khác hẳn** dữ liệu BI dạng bảng: lồng nhau sâu, độ sâu thay đổi theo từng lượt, nặng ngôn ngữ tự nhiên
- Cần chính sách retention/PII **riêng** — một trajectory thường mang PII đi xuyên qua nhiều tool call
- Có thể tái sử dụng làm dữ liệu huấn luyện cho thế hệ agent kế tiếp
- Kết quả: pipeline nuôi agent không còn là một đường thẳng Sources → Agent — nó là một **vòng tròn**

Nguồn: nghiên cứu agent-trajectory 2025–2026 (EMNLP 2025 demo toolkit; DataGovBench, arXiv 2512.04416; Trajectory Guard, arXiv 2601.00516).

Layer	Tools 2026
Ingestion	Airbyte — 620 connector (catalog, 2026-08), gồm hạng mục mới “Agent Connectors” (hơn 50, theo Airbyte); Fivetran — từ 2026-06-01 cùng công ty với dbt Labs ; warehouse-native: Lakeflow Connect, Openflow
Transform	dbt Labs (nay = Fivetran); SQLMesh/Tobiko — do Fivetran mua lại (2025-09-03)
Storage / Compute	Lakehouse (Databricks, Snowflake) — kiến trúc đã hội tụ, cạnh tranh chuyển sang execution/pricing
Orchestration	Apache Airflow 3.x; Prefect — đang mua lại Dagster Labs (2026-07-13)
Observability	Monte Carlo (nay montecarlo.ai); Elementary

Toàn ngành pivot marketing sang “AI agent” cùng cửa sổ 2025–2026 — dấu hiệu họ nhìn thấy cùng một khách hàng mới bạn đang xây.

Benn Stancil, “*The Data OS*” (2021-09-03): Y Combinator 2017 rót vốn 15 công ty analytics/data-engineering, 2021 là 100 — kêu gọi một lớp hợp nhất kiểu Android, gọi tên chính **dbt**. Góc nhìn giảng viên: bằng chứng ủng hộ, chưa phải kết luận.

Airbyte catalog 2026-08 · Fivetran/dbt Labs 2026-06-01 · Qlik 2026 · Databricks 2025-06-12 · Snowflake 2025-05 · dagster.io 2026-07-13. Chi tiết từng layer: §3–§5, §9–§10, §13.

ETL — Transform Trước Khi Load

- Extract → **Transform** (trên compute riêng) → Load
- Schema cố định trước khi dữ liệu chạm kho lưu trữ (schema-on-write)
- Phù hợp: pipeline BI truyền thống, nơi dữ liệu phải “sạch” trước khi lưu vì lý do compliance
- Tool lịch sử hay được nhắc: **Talend** — nay bán dưới tên **Qlik Talend Cloud**, không còn là công ty độc lập (Qlik, 2026)

ELT — Load Trước, Transform Sau

- Extract → Load (dữ liệu thô) → **Transform** (bằng SQL, ngay trong warehouse)
- Giữ nguyên bản gốc (schema-on-read) — transform lại bất cứ lúc nào, không cần nạp lại từ nguồn
- Phù hợp: lakehouse hiện đại, chuẩn bị corpus cho RAG/agent
- **dbt** (Transform) + **Fivetran** (Ingestion) — nay một công ty, xem bảng trang trước

Phần lớn AI/LLM team 2026 đi **ELT**: giữ raw để transform lại khi đổi chunking (chunking: Ngày 7), và compute-trong-warehouse rẻ hơn tăng transform riêng.

Mới 2026: warehouse vendor nuốt luôn bước ingestion — Lakeflow Connect, Openflow (chi tiết + ngày GA ở §16 Frontier).

Câu hỏi 2026 không còn là “Fivetran hay Airbyte” mà “hay dùng luôn ingestion tool vendor đã ship sẵn?” Đánh đổi: **lock-in**.

Batch — Xử Lý Theo Lô

- Chạy theo lịch: hourly, nightly, hoặc theo trigger thủ công
- Throughput cao, chi phí thấp hơn cho khối lượng lớn
- Độ trễ: từ vài phút đến vài giờ giữa lúc sự kiện xảy ra và lúc dữ liệu sẵn sàng

Streaming — Xử Lý Liên Tục

- Sự kiện tới đâu, xử lý tới đó — không có “cửa sổ” chờ giữa hai lần chạy
- Độ trễ thấp (giây hoặc dưới giây), nhưng hạ tầng và vận hành phức tạp hơn hẳn
- Rủi ro: lỗi lan truyền tức thì — không có khoảng nghỉ giữa hai lần chạy để phát hiện trước

Workload của agent có hình dạng khác hẳn BI — agent “phân nhánh và query hàng trăm lần mỗi giây” (Jordan Tigani, MotherDuck, “*Future Casting the Modern Data Stack*”, 2026-03-23). Batch chạy theo lịch không theo kịp: đơn hàng vừa hủy, tồn kho vừa hết, người dùng vừa đổi ý giữa hội thoại.

Câu hỏi thiết kế: corpus của RAG agent bạn cập nhật mỗi giờ hay mỗi giây?

Lưu ý: Chọn sai chiều batch/streaming là quyết định kiến trúc phải chốt trước dòng code ingestion đầu tiên — agent sẽ trả lời bằng dữ liệu đã nguội.

Luật chơi

Với mỗi tình huống dưới đây, xác định: (1) ETL hay ELT, (2) Batch hay Streaming. Không có đáp án trên slide — thảo luận theo bàn.

1. Một fintech startup cần dashboard doanh thu sẵn sàng lúc 8 giờ sáng mỗi ngày cho CFO.
2. Agent chăm sóc khách hàng phải trả lời ngay “còn hàng không?” bằng đúng tồn kho hiện tại.
3. Team data science muốn thử 5 chiến lược chunking khác nhau trên cùng một tập tài liệu trong tuần này.
4. Ngân hàng phải lưu giao dịch đã qua kiểm duyệt, không được sửa lại sau khi đã lưu, vì lý do compliance.

Ingestion — Thu Thập Data Từ Nhiều Nguồn

Kết nối nguồn đa dạng vào pipeline một cách đáng tin cậy

03

Nguồn tĩnh (at rest)

- **Structured:** bảng quan hệ trong database nội bộ, data warehouse export, file CSV/Excel
- **Unstructured / semi-structured:** PDF, Word, email, tài liệu trên Google Drive/SharePoint — đây chính là kho corpus bạn nạp vào vector store ở Ngày 8
- Ingest theo lịch (batch): full refresh hoặc CDC định kỳ

Nguồn động (streaming / event)

- **Event streams:** Kafka topic, webhook, log ứng dụng, sự kiện IoT — data đến liên tục, không theo lịch cố định
- API cần được **poll** liên tục hoặc đăng ký webhook để nhận sự kiện gần thời gian thực
- Consumer phải luôn chạy — không phải kiểu "chạy rồi xong" như batch job
- Nếu agent cần biết trạng thái đơn hàng gần real-time, nó phải đọc từ nguồn dạng này, không phải bảng batch nạp mỗi đêm

Change Data Capture (CDC) — Bắt các thay đổi (insert/update/delete) trên nguồn ngay khi xảy ra, thay vì định kỳ quét lại toàn bộ dataset.

- **Full scan / full refresh:** quét lại toàn bộ bảng mỗi lần chạy — chi phí tăng tuyến tính theo kích thước bảng, càng chạy càng chậm khi nguồn lớn dần
- **CDC:** chỉ lấy phần đã đổi — chi phí phụ thuộc *lượng thay đổi*, không phụ thuộc kích thước toàn bảng
- Ứng dụng vào agent: bảng nội bộ có hàng triệu dòng nhưng chỉ một phần nhỏ thay đổi mỗi phút — CDC chỉ đồng bộ phần thay đổi đó; full scan vẫn phải đọc lại toàn bộ bảng mỗi lần
- Full scan còn có nguy cơ khác: dữ liệu bị sửa rồi sửa lại giữa hai lần quét sẽ chỉ thấy được trạng thái cuối cùng, mất dấu các bước trung gian

Log-based

- Đọc trực tiếp **transaction log** / **WAL** / **binlog** — thấy mọi thay đổi ngay khi commit
- Tool phổ biến nhất: **Debezium**, chạy trên Kafka Connect
- Không tạo thêm tải **query** lên nguồn — đọc log, không đọc lại bảng
- Thấy cả thay đổi trung gian (vd. insert rồi delete) giữa hai lần sync

Query-based

- Poll định kỳ bằng cột mốc: `WHERE updated_at > last_sync`
- Không cần quyền truy cập log — dễ triển khai hơn
- Tạo tải **query lặp lại** lên nguồn mỗi chu kỳ poll
- Bỏ sót thay đổi tạo-rồi-xoá xảy ra hoàn toàn giữa hai lần poll

- **Trước (kiểu 2020):** cron hằng ngày `SELECT * FROM table` → S3 → import Snowflake; phải dựng thêm read-replica chỉ để giảm tải cho việc export
- **Sau:** CDC incremental sync (Kafka Connect + CDC service tự viết) qua Amazon MSK, nạp Snowflake bằng `MERGE`, orchestrate bằng AWS Step Functions

Trước → Sau (Figma Engineering, 2025-05-29)

- Thời gian sync: 6+ giờ (nhiều ngày với bảng lớn nhất) → **3 giờ** mặc định
- Freshness: stale 30+ giờ → **3 giờ** (cấu hình được xuống phút)
- Xử lý được bảng lớn gấp **10 lần**
- Tiết kiệm hàng triệu USD/năm

“Building bespoke solutions proved more cost-effective — proprietary/vendor tools would have cost 5–10x more.” (Figma Engineering, 2025-05-29)

Tool	Con số / định vị	Ghi chú
Airbyte	620 connector + 50+ Agent Connectors (mới)	Open-source, tính đến 2026-08
Fivetran	Mốc cuối chính thức: 500+ connector (2024-03)	Số 600–900+ hiện hành là marketing/third-party chưa kiểm chứng
dlt (dlthub)	Code-first — viết connector bằng Python	Mặc định 2026 khi cần build connector tùy biến
Stitch	Không còn công bố số connector riêng	Nay là sub-brand legacy của Qlik — không khuyến nghị dự án mới
Singer	Hai loại: taps (extract) + targets (load)	Không chỉ “taps” — thiếu target thì không load đi đâu được

Airbyte catalog (2026-08); Fivetran press release (2024-03); dlthub blog (2026-05-19); singer.io; stitchdata.com — connector count là số vendor tự báo, luôn gần ngày khi trích

Lưu ý: “Zero-ETL” không có nghĩa là hết data engineering. Theo tài liệu chính AWS: managed, một chiều, read-only, **không transform trong luồng**; CDC replication cho vài cặp source→target được hỗ trợ chính thức; cần primary key (AWS Redshift docs).

Số liệu tự báo cáo của dlt qua telemetry (2026-05-19) — gần nhãn vendor, chưa audit độc lập

- Pipeline mới tạo mỗi tháng: **2.400** (1/2025) → **81.000** (1/2026) — khoảng 34 lần
 - Tỷ lệ pipeline do **agent viết**: 5% → 91% (theo dlt)
 - Hơn **10.000** công ty chạy dlt trong production (theo dlt)
-
- Đây là con số dlt tự đo, chưa được kiểm toán độc lập — nhưng hướng đi là thật: pipeline ingest bạn dùng hôm nay có thể do một agent khác viết ra, không phải người
 - Câu hỏi cho bạn: nếu agent tự viết pipeline ingest cho chính nó, ai review đoạn code đó trước khi nó chạy lúc 2 giờ sáng?

Rate limiting & backpressure

- Nguồn giới hạn số request/giây — vượt ngưỡng bị từ chối (HTTP 429)
- Backpressure: khi consumer xử lý chậm hơn producer, cần buffer hoặc tín hiệu pause để nguồn tự giảm tốc
- Không xử lý backpressure → tràn bộ nhớ hoặc rớt record âm thầm
- Retry, backoff, DLQ và điều kiện idempotency: cơ chế đầy đủ ở (§13)
- Agent gọi API bị rate-limit không có backoff → request biến mất, corpus thiếu data mà không log nào báo lỗi

- **Full refresh**: load lại toàn bộ dataset mỗi lần chạy — đơn giản, nhưng chi phí tăng tuyến tính theo kích thước nguồn
- **Incremental**: chỉ load phần thay đổi kể từ lần chạy trước, dựa trên **watermark** — mốc đánh dấu “đã load tới đâu” (thường là timestamp hoặc offset)
- **Late-arriving data**: record có timestamp cũ nhưng đến pipeline trễ (vd. thiết bị offline rồi mới sync) — nếu watermark đã “chốt” mốc trước đó, record này có thể bị bỏ sót vĩnh viễn

Watermark — Giá trị mốc (timestamp hoặc offset) đánh dấu ranh giới giữa dữ liệu “đã ingest” và “chưa ingest” trong một lần chạy incremental.

Nếu tài liệu chính sách cập nhật trễ hơn watermark của lần chạy gần nhất, nó sẽ không bao giờ vào corpus của agent — agent tiếp tục trả lời theo chính sách cũ, tự tin và sai.

- Làm việc theo cặp, dùng chính agent bạn đã xây ở Ngày 8 hoặc Ngày 9
- Liệt kê **ít nhất 3 nguồn** agent đó đang đọc: 1 internal DB, 1 external API, 1 kho tài liệu (PDF/Drive)
- Điền bảng dưới cho từng nguồn — cột cuối là trọng tâm của bài tập

Nguồn	Loại	Cơ chế ingest	Điểm fail <i>âm thầm</i>
Bảng orders (internal DB)	Structured	CDC / query-based?	...
API bên ngoài (vd. payment)	Structured (JSON)	Poll theo lịch? Webhook?	...
Kho chính sách (PDF/Drive)	Unstructured	Upload thủ công?	...

8 phút, thảo luận cặp. Nhắc lại: một job ingestion chết lúc 2 giờ sáng không tạo lỗi ở bất kỳ đâu người dùng nhìn thấy — agent chỉ lặng lẽ trở nên ít nói hơn và cũ hơn.

Transform — Làm Sạch & Chuẩn Hóa Data

Biến raw data thành data agent tin được

04

Vấn đề dữ liệu

- **Missing values** — ô trống, null giả dạng chuỗi ("N/A", "-")
- **Outliers** — giá trị bất thường, có thể là lỗi nhập liệu hoặc tín hiệu thật
- **Duplicates** — cùng một bản ghi xuất hiện nhiều lần, đôi khi lệch định dạng nhỏ
- **Wrong formats** — ngày tháng dd/mm/yyyy vs mm/dd/yyyy, số dùng dấu phẩy vs dấu chấm
- **Encoding** — file lưu sai bảng mã (Windows-1258, ISO-8859-1) làm tiếng Việt vỡ chữ

Điều này làm gì với agent của bạn

- Missing value không xử lý → agent suy diễn từ context rỗng → trả lời như thể có dữ liệu
- Duplicate trong corpus RAG → retrieval trả cùng một đoạn nhiều lần, lãng phí context window
- Outlier không lọc → agent trích dẫn con số vô lý làm “fact”
- Encoding vỡ → chunk tiếng Việt ra ký tự lạ → embedding vô nghĩa

Nguyên tắc

Transform quyết định agent có **tin được** data hay không — không chỉ là “dọn cho đẹp”.

Vấn đề: Unicode NFC vs NFD

- Tiếng Việt có dấu (diacritics) có **hai dạng Unicode khác nhau**:
 - ▷ **NFC** (composed) — 1 ký tự đã ghép sẵn: “á” = U+00E1
 - ▷ **NFD** (decomposed) — 2 ký tự: “a” + dấu sắc kết hợp
- Hai chuỗi **trông giống hệt nhau** khi hiển thị nhưng **khác byte**
- Nguồn gốc: macOS thường lưu NFD, Windows/web thường lưu NFC, data scrape trộn cả hai
- Exact-match/dedup so byte-for-byte → “á”(NFC) ≠ “á”(NFD) → **fail silently**, agent báo “không tìm thấy” dù data có sẵn

Vấn đề: PII

- PII (SĐT, CCCD, email, địa chỉ) trong document raw **đi thẳng vào vector store** nếu không bị chặn tại transform
- PII đã lọt vào embedding → agent có thể **trích dẫn nguyên văn** nó cho bất kỳ user nào hỏi đúng câu
- Mask/redact PII là một **bước transform**, cùng chỗ với cleaning — không phải việc của prompt/guardrail sau này

Lưu ý: PDPL — Luật số 91/2025/QH15, hiệu lực **2026-01-01**: dữ liệu cá nhân cần xử lý có mục đích rõ ràng, kèm nghĩa vụ xóa/ẩn danh khi hết mục đích. Chi tiết pháp lý và stack 3 tầng ở §11.

Fix bắt buộc trước khi embed

Chuẩn hóa text về **một dạng** (khuyến nghị NFC) + mask/redact PII — cả hai là bước **transform**, không phải việc của prompt/guardrail sau. Kèm: collapse whitespace, strip HTML, language detection.

```
-- models/staging/stg_documents.sql
with source as (
    select * from {{ ref('raw_documents') }}
)

select
    doc_id,
    trim(title) as title,
    -- Postgres/Snowflake/DuckDB syntax
    created_at::date as doc_date,
    lower(language_code) as language_code
from source
where doc_id is not null
```

- `{{ ref(...) }}` là idiom chuẩn dbt — code, review được, chạy lại được (không phải script tay chạy một lần)
- Mỗi model là **một bước transform có version control**, giống code của bạn

Lưu ý: `created_at::date` là cú pháp Postgres/Snowflake/DuckDB, **không phải ANSI SQL**. Trên BigQuery phải dùng `CAST(created_at AS DATE)` hoặc `DATE(created_at)`.

	Data tests (cổ điển)	Unit tests (từ dbt v1.8)
Kiểm cái gì	Data thật trong warehouse, sau khi model materialize	Logic SQL transformation, trên fixture tĩnh, trước khi materialize
Trả lời câu hỏi	“Data production hiện có khỏe không?”	“Logic transform của tôi có đúng không?”
Ví dụ	not_null, unique, relationships	Input rows X → expect output rows Y (kiểu TDD)
Chạy bằng	dbt test	dbt test (cùng lệnh)

dbt docs, docs.getdbt.com/docs/build/unit-tests

- **Document extraction bằng LLM vision:** chi phí mỗi trang đã giảm mạnh (dưới 1 xu đến vài xu/trang, xu hướng thật 2024→2026) — nhưng **con số accuracy lưu hành là vendor tự báo cáo**, chưa có benchmark độc lập. Pattern production hội tụ: OCR rẻ cho layout ổn định, dành LLM cho phần messy/scan xấu
- **LLM-assisted schema mapping:** kết quả nghiên cứu **hỗn hợp**, không phải chiến thắng sạch — một phương pháp mới thắng baseline trên bộ dữ liệu này nhưng thua trên bộ khác (arXiv 2505.24716, 2025-05-30)
- **Entity resolution với embedding + LLM:** lĩnh vực đang nghiên cứu tích cực 2025, **chưa chuẩn hoá** — pattern production là hybrid: vector similarity sinh candidate, LLM làm judge ở mức cặp/cụm

Pattern chung đáng nhớ

“LLM proposes, code disposes” — để LLM đề xuất, nhưng luôn bọc validation/post-processing bằng code ở ngoài. Đừng để LLM tự quyết định trong pipeline mà không ai kiểm.

Transform cho BI

- Đích đến: bảng có schema cố định, join được, aggregate được
- Chuẩn hoá kiểu dữ liệu, khoá ngoại, grain của bảng
- Lỗi transform → dashboard vỡ, số sai lệch rõ — fail **loud**
- Test: data test kiểu `not_null/unique` trên cột

Transform cho AI/RAG

- Đích đến: text sạch, chuẩn Unicode, đã bóc PII — sẵn sàng **chunk** rồi embed (chunk/embedding: **Ngày 7**)
- Chuẩn hoá encoding, whitespace, ngôn ngữ — không phải kiểu dữ liệu cột
- Lỗi transform → agent vẫn trả lời trôi chảy nhưng sai — fail **silent**
- Test: khó viết assertion kiểu `not_null` cho “đoạn text này còn đúng nghĩa không?”

Bối cảnh: bạn nhận một bảng nhỏ “documents” chuẩn bị nạp vào vector store cho agent hỗ trợ khách hàng. Bảng có sẵn các lỗi sau, trộn lẫn:

- ☐ Một dòng phone = "N/A", một dòng phone để trống thật
- ☐ Hai dòng nội dung giống hệt nhau nhưng một dòng tên khách hàng gõ dạng NFC, dòng kia gõ dạng NFD
- ☐ Một dòng created_at = "05/08/2026" (không rõ dd/mm hay mm/dd)
- ☐ Một dòng chứa số CCCD ngay trong nội dung mô tả
- ☐ Một dòng có tag
 và sót lại từ HTML

Yêu cầu: viết các bước transform (giả code hoặc dbt model) xử lý **từng** vấn đề trên, và giải thích ngắn gọn: nếu bỏ qua bước nào, agent sẽ hỏng ở đâu?

Storage & Serving Cho Agent

Data nằm ở đâu, và agent lấy nó ra thế nào

Loại kho	Dùng để làm gì	Agent chạm vào nó thế nào
Data Warehouse	Structured, schema-on-write, tối ưu SQL/BI, aggregate & report	Qua SQL tool-call cho structured lookup — hiếm khi là nguồn RAG trực tiếp
Data Lake	Raw files trên object storage, schema-on-read, rẻ, mọi định dạng	Vùng hạ cánh của pipeline — agent hầu như không bao giờ đọc thẳng
Lakehouse	Table format (Iceberg/Delta/Hudi) đặt ACID + schema evolution lên trên data lake	Nơi curated table phục vụ cả BI lẫn context cho agent
Vector Store	Lưu embedding, phục vụ semantic similarity search	Đây là nơi RAG agent đọc trực tiếp mỗi query

- **AWS S3 Vectors**: preview từ 7/2025, **GA ngày 2025-12-02** tại re:Invent 2025 (AWS, 2025-12)
- Giới hạn capacity sau GA: **2 tỷ vector/index**, 10.000 vector index/bucket (AWS, 2025-12)
- Latency/throughput sau GA: khoảng **100ms hoặc thấp hơn** cho truy vấn “frequent”, dưới 1 giây cho “infrequent”, tối đa 100 kết quả/truy vấn, write throughput tới 1.000 PUT TPS (AWS, 2025-12)
- Theo AWS **tự báo cáo** tại re:Invent 2025 (không có audit độc lập): hơn 250.000 vector index và hơn 40 tỷ vector đã ingest trong khoảng 4 tháng preview
- Theo marketing của **chính AWS**: “tiết kiệm tới 90% chi phí” — so với một alternative không nêu tên

Tranh luận kiến trúc còn mở (2026)

Vector-store-như-một-storage-class-của-S3 có thật sự đơn giản hoá kiến trúc RAG pipeline của bạn, hay đẩy workload latency thấp quay lại vector DB chuyên dụng (Pinecone, Weaviate)? Giới phân tích chưa đồng thuận. *Thuật toán ANN bên dưới và cách chọn engine: Ngày 7.*

Với agent của bạn: vector store bạn chọn quyết định agent chờ bao lâu trước khi trả lời — đó là latency budget, không phải chi tiết ẩn dưới hạ tầng.

- **Iceberg 1.11.0** (tag 2026-05-20, công bố 2026-05-27): REST-catalog server-side scan planning, table encryption tích hợp, File Format API, SQL UDF spec, hỗ trợ mặc định Spark 4.1 + Flink 2.1 (Apache Iceberg, 2026-05)
- **Delta** ↔ **Iceberg hội tụ**: Databricks mua **Tabular** — chính đội sáng lập Iceberg (Ryan Blue, Daniel Weeks) — công bố 2024-06-04. **UniForm** sinh song song metadata Iceberg + Hudi trên cùng một bản Parquet; **Delta Kernel** (Delta Lake 4.0) cho engine ngoài Spark đọc/ghi Delta nhất quán (Databricks, 2024–2026)
- Benchmark trung lập giữa Iceberg/Delta/Hudi gần như không tồn tại — ngoại lệ **LST-Bench** (arXiv 2305.01120, peer-reviewed tại PACMMOD 2024, không do tác giả của format nào viết). Hầu hết so sánh khác lưu hành đều do chính một trong ba vendor công bố.

Với agent của bạn

Bạn không cần thắng-thua giữa 3 format để bắt đầu. Chọn engine bạn dùng hỗ trợ tốt nhất — agent không quan tâm format nào nằm dưới, miễn curated table đọc được. *Kiến trúc lakehouse đầy đủ, catalog wars, spec chi tiết: Ngày 18.*

Bronze (raw, giữ để audit/replay) → **Silver** (dedupe + chuẩn hoá schema, check chất lượng chạy ở đây, §6) → **Gold** (aggregate, sẵn sàng phục vụ — BI dashboard đọc từ đây, và pipeline nạp vector store cũng đọc từ đây).

Agent đọc từ Gold. Đừng bao giờ trở RAG ingestion thẳng vào Bronze.

Pipeline cam kết gì

- Corpus phản ánh nguồn tính đến lần **ingestion run gần nhất** thành công
- Daily hash-based re-ingestion check là “đủ tốt” cho phần lớn corpus
- Event-driven / webhook-triggered ingestion cho nguồn thực sự real-time

Vector store cam kết gì

- Index phản ánh lần **upsert/reindex thành công gần nhất** — không phải lần document nguồn thay đổi
- Độ trễ giữa “ingest xong” và “query thấy version mới” phụ thuộc lịch reindex, không phải SLA của nguồn
- “Corpus hiện hành” là một **timestamp vận hành cụ thể**, không phải một lời hứa chung chung

Phải re-embed toàn corpus

- **Đổi embedding model** → vector space mới, embedding cũ và mới không so sánh được
- **Đổi chiến lược chunking** → ranh giới chunk khác, offset cũ vô nghĩa

Incremental upsert là đủ

- Chỉ thêm/sửa document, model và chunking **không đổi** → upsert vector mới, giữ nguyên phần còn lại
- Đây là ca phổ biến nhất trong vận hành hàng ngày

Né chi phí re-embed: Drift-Adapter (arXiv 2509.23471, Harshil Vejendla, nộp 2025-09-27)

Học một **transform nhẹ** ánh xạ embedding của query mới vào không gian embedding cũ, huấn luyện trên một mẫu nhỏ các cặp (old, new) embedding — thay vì re-embed toàn bộ corpus. Ba biến thể: Orthogonal Procrustes, Low-Rank Affine, Residual MLP. Kết quả tác giả báo cáo: phục hồi **95–99% Recall@10 và MRR** so với một lần re-embed đầy đủ, thêm **dưới 10 microsecond** latency mỗi query, giảm chi phí recompute **hơn 100 lần**.

Lưu ý: Preprint một tác giả, chưa peer review, chưa ai replicate — dùng để hiểu kỹ thuật, đừng trích như kết quả đã kiểm chứng.

- ✓ **Dual-index serving** — build index mới song song với index cũ, cắt traffic đọc sang index mới sau khi validate xong
- ✓ **Gradual traffic shift** — route dần một phần trăm query sang index mới, theo dõi, tăng tỷ lệ theo thời gian
- ✓ **Maintenance window** — chấp nhận downtime ngắn, swap tại một mốc cố định; chỉ hợp lý khi SLA freshness cho phép

Với nửa lớp học phần mềm

Đây đúng là **blue-green deployment** — chỉ khác là bạn đang deploy một phiên bản mới của *data*, không phải code. Chọn chiến lược nào phụ thuộc SLA freshness của bạn (§7 định nghĩa SLA đó thành số cụ thể).

Data Quality — 6 Dimensions

Đo chất lượng data trước khi nó đến tay agent

- ☒ **Completeness** — thiếu dữ liệu?

Check: % NULL trên field bắt buộc → agent thiếu context, bỏ sót thông tin khi trả lời.

- ☒ **Accuracy** — đúng thực tế?

Check: đối chiếu mẫu với system of record → agent trích dẫn lệch nhưng vẫn trả lời rất tự tin.

- ☒ **Consistency** — nhất quán giữa hệ thống?

Check: customer.email khớp giữa CRM và warehouse → agent trả lời khác nhau tùy chunk nào được retrieve.

- ☒ **Timeliness** — có sẵn đúng lúc?

Check: freshness — độ trễ từ event đến khi row có mặt trong warehouse → agent trả lời bằng thông tin đã cũ (stale corpus).

- ☒ **Validity** — đúng format/rule?

Check: order_status nằm trong enum, email khớp regex → parse sai hoặc crash ở bước ingest.

- ☒ **Uniqueness** — không trùng lặp?

Check: doc_id / primary key không có bản sao → retrieval trả về cùng một đoạn nhiều lần, lãng phí context window.

*Taxonomy truyền thống, vendor-neutral (DAMA-DMBOK / ISO-25012-adjacent): đánh giá **một dataset tại một thời điểm**. Khác “5 Pillars” (§9) — framework **thương hiệu** của một vendor, giám sát **cả một pipeline theo thời gian**. Không tool nào chạy một lần rồi xong — đặt check ở đâu trong pipeline là chuyện của §7.*

- **Profiling là gì:** khảo sát dữ liệu THẬT trước khi viết bất kỳ rule nào — tỷ lệ NULL, cardinality, min/max, phân bố giá trị, kiểu dữ liệu suy ra từ từng cột.
- **Tại sao trước khi assert:** nếu bạn đoán rule mà không nhìn dữ liệu thật (vd “status chỉ có 5 giá trị”), rule sẽ vỡ ngay khi gặp giá trị thứ 6 — không phải vì dữ liệu sai, mà vì rule đoán sai.
- **Profiling đo chính 6 dimension ở trên** — vd % NULL cho Completeness, cardinality cho Uniqueness, phân bố giá trị cho Validity.
- **Có sẵn trong tooling:** PyDeequ có *Constraint Suggestion* — tự profile dữ liệu rồi đề xuất constraint (PyDeequ 1.6.0, 2026-07-08).

Với corpus RAG của bạn: nếu bạn không profile trước — vd không biết có bao nhiêu chunk rỗng, độ dài phân bố ra sao — bạn sẽ viết expectation “đoán mò” và bỏ sót đúng loại lỗi làm agent trả lời sai.

- **Ý tưởng:** rule sống trong CODE, review trong pull request như bất kỳ code nào khác — không phải config hay dashboard tách rời.
- **GX Core vs GX Cloud:** Core là engine open-source code trực tiếp vào pipeline; Cloud là lớp UI hosted bên trên cho team.
- GX Core 1.0 GA **2024-08-22**; bản hiện tại **1.19.1 (2026-07-24)**, **Python 3.10–3.13** (GX docs/PyPI, 2026-07).
- Từ 1.0, config gần như hoàn toàn Python-first: *“All configurations except credentials are now in Python.”*

Vì sao quan trọng với agent: pipeline RAG không có expectation nào canh, cột content rỗng/NULL âm thầm lọt qua — agent không crash, nó chỉ trả lời sai một cách tự tin.

Lưu ý:

1. `batch.validate(expectation).success` **VẪN CHẠY** trên 1.x — `ExpectationValidationResult` hỗ trợ cả `.success` lẫn `["success"]`. **KHÔNG** phải dòng bị hỏng.
2. `gx.get_context()` **VẪN hợp lệ** trong 1.x — chỉ những gì đến SAU nó (`Validator`, `batch_request`, block-style Data Sources) mới bị xoá.
3. Đúng: *“hỏng trên GX 1.0+ (từ 2024-08-22)”* — **KHÔNG PHẢI** “mọi bản cài sau 8/2024”. `pip install great-expectations==0.18.22` hôm nay vẫn chạy code cũ.

Đề sau khi đã quen object model: production gom nhiều expectation vào một `ExpectationSuite`, gán `ValidationDefinition` để chạy lặp lại tự động (`context.suites.add(...)`, `context.validation_definitions.add(...)`).

```
import great_expectations as gx
import pandas as pd

context = gx.get_context()
# Data Source -> Data Asset -> Batch Definition -> Batch
data_source = context.data_sources.add_pandas(name="docs_source")
data_asset = data_source.add_dataframe_asset(name="docs_asset")
batch_def = data_asset.add_batch_definition_whole_dataframe("docs_batch")

df = pd.read_csv("docs.csv")
batch = batch_def.get_batch(batch_parameters={"dataframe": df})

# Expectation is now a standalone OBJECT, not a method on batch
not_null = gx.expectations.ExpectColumnValuesToNotBeNull(column="content")

# Batch validates AGAINST the Expectation -- direction is reversed
print(batch.validate(not_null).success)  # True / False
```

Cột content của một corpus tài liệu — đúng thứ bạn nạp cho RAG ở Ngày 8 (GX docs, 2026-07).

Tool	Là gì	Dùng khi nào
Great Expectations	Quality-as-code, open-source, expectation là Python object	Rule cần review trong PR như code
Soda	Chuyển từ SodaCL sang Data-Contracts-first (4.0, GA 2026-01-28)	Rule gắn liền data contract (§7), không phải check rời
Elementary	dbt package dbt-data-reliability, dbt-native	Đã có dbt project, muốn dùng meta-data dbt có sẵn
dbt tests	Test built-in của dbt (not_null, unique, relationships)	Check nhanh ngay trong transform layer
PyDeequ	Spark-native, metrics + Constraint Suggestion (1.6.0, 2026-07)	Data quality quy mô Spark
Pandera	“pydantic cho dataframe” (0.32.1, 2026-06)	Schema Python-first, đa backend (pandas, Polars, PySpark...)

PyPI & vendor sites, 2026-06/08 — GX 3.10–3.13, PyDeequ 3.9–3.12 (giao nhau: 3.10–3.12).

Quality Gates & Data Contracts

Chặn data xấu ở đâu, và ai chịu trách nhiệm

1. **Ingestion boundary** — record vừa vào hệ thống: schema đúng không, field bắt buộc có null không
2. **Post-transform** — sau clean/dedupe/join: giá trị còn hợp lệ sau biến đổi không
3. **Pre-embed** — trước khi sinh embedding: doc đã stale, trùng lặp, hay mâu thuẫn với version mới hơn chưa
4. **Pre-serve** — ngay trước khi agent truy vấn index: câu trả lời có đang dựa trên data đã bị đánh dấu lỗi không

Sớm hay muộn?

Gate càng **sớm** → rẻ hơn (chưa tốn compute transform/embed) nhưng **ít context** — bạn chưa biết doc này sẽ trả lời câu hỏi nào.

Gate càng **muộn** → tốn hơn nhưng bắt được lỗi mà chỉ ngữ cảnh đầy đủ mới lộ ra, ví dụ hai tài liệu mâu thuẫn nhau.

Gate = cách bạn biến silent failure (agent trả lời sai, không báo lỗi) thành loud failure (pipeline dừng, có người phải nhìn) — có chủ đích.

Fail-Closed

- Check fail → **pipeline dừng**, không update index
- Agent tiếp tục serve data **cũ nhưng đúng**
- Chọn khi: trả lời cũ rẻ hơn trả lời sai — ví dụ chính sách pháp lý, số liệu tài chính

Fail-Open

- Check fail → pipeline **log rồi chạy tiếp**, vẫn update index
- Agent serve data **mới nhưng có thể sai**
- Chọn khi: data cũ gây hại nhiều hơn data đáng ngờ — ví dụ giá real-time, tồn kho

Contract cho BI pipeline hỏi

Schema có đúng không? Giá trị có nằm trong domain cho phép không?

Contract cho agent corpus hỏi thêm

Chunk này còn tươi không? Có bị mâu thuẫn bởi phiên bản mới hơn không? Embedding model version nào sinh ra nó?

Metadata tối thiểu gắn vào mỗi chunk (đồng thuận practitioner nhiều nguồn, 2025–2026): embedding model version, preprocessing hash, text checksum, index version, chunking config, `version + indexed_at` → cho phép rollback và audit “agent nhìn thấy gì tại thời điểm query N”.

Lưu ý: Không bao giờ trộn embedding từ hai model version / preprocessing pipeline khác nhau trong cùng một index — *mixed embeddings = chaotic geometry*. Drift check tự động hàng tuần; A/B test chất lượng retrieval trước khi roll out model upgrade.

Một data contract đặc tả:

- **Schema** — field nào, kiểu gì, required/optional
- **Semantics** — field nghĩa là gì (vd: active_user tính từ đâu)
- **SLA** — cam kết freshness, uptime, latency
- **Ownership** — ai chịu trách nhiệm khi vi phạm

*Shift-left: kiểm tra hợp đồng producer–consumer **trước** khi data chảy vào pipeline, không phải sau khi agent đã trả lời sai. ODCS chi tiết + OpenLineage: sâu hơn ở Ngày 27.*

ODCS / Bitol — chuỗi nguồn gốc:

1. Template nội bộ **PayPal**
2. **AIDA User Group** tiếp quản, duy trì từ v2.2.0
3. **2023-11-30: AIDA UG + Linux Foundation AI & Data** → lập ra **Bitol**
4. Bitol quản trị **ODCS v3.1.0** (12/2025, Apache 2.0)

(Bitol history.md, 2026)

Arc 3 bước: mandate (Midas certification) → tooling (Wall, 2021) → incentive (DQ Score, 2023).

Wall (2021-08-04)

Framework Python trên **Apache Airflow**; check định nghĩa bằng **YAML**, không cần code. Hàng trăm pipeline chạy hàng nghìn check Wall mỗi ngày; migrate sang Wall giảm **hơn 70%** code DAG. Mọi pipeline critical business/tài chính đều qua Wall; check thực thi qua **Presto**, kết quả publish thành Kafka event.

DQ Score (2023-11-28)

Điểm **0–100** trên 4 chiều: Accuracy 40, Stewardship 30, Reliability/Timeliness 15, Usability 15. “...we could no longer rely on enforcement... we instead needed to rely on incentivization.” Chỉ data đã được **Midas certify** mới đạt điểm > 90 — certification là điều kiện, không phải kết quả.

Đây là case của pipeline BI, không phải agent/RAG — nhưng là enforcement-gate được tài liệu hoá tốt nhất hiện có. Cùng logic (mandate → tooling → incentive) áp dụng khi bạn xây gate cho corpus nuôi agent. (Airbnb Engineering, 2021-08-04 & 2023-11-28)

“**Data contracts are a people problem dressed in YAML syntax.**” — Chad Sanderson, blog Gable.ai (2026) — **lưu ý: đây là vendor content**, Gable bán chính sản phẩm lineage-đọc-từ-code cạnh tranh với cách tiếp cận data contract thủ công.

Pattern thất bại kinh điển mà bài viết mô tả:

- Một chuẩn công ty-wide được đề xuất, spec được viết ra — 6 tháng sau chỉ còn contract của chính người đề xuất (không ai khác áp dụng, **không phải** do bị cắt ngân sách)
- Thoả thuận mục ruỗng vì tổ chức tái cấu trúc nhanh hơn tốc độ contract được cập nhật
- Rải contract lên hàng nghìn interface thay vì tập trung vào ranh giới trọng yếu → thất bại

Một contract, giống một gate, chỉ hoạt động nếu có người enforce nó — công cụ không tự làm thay (Gable.ai blog, 2026).

Từ Data-issue Đến Agent-symptom

Agent sai kiểu gì — và lần ngược về tầng data

08

Triệu chứng người dùng thấy	Nguyên nhân gốc ở tầng data	Tín hiệu observability
Trả lời tin cũ, rất tự tin	Ingestion fail âm thầm; freshness SLI chưa set	Freshness (max age) + Volume = 0 (§9)
Trích chính sách không tồn tại	Corpus mâu thuẫn, hoặc thiếu tài liệu	Không tín hiệu nào bắt được (§15)
Cùng câu hỏi, 2 số khác nhau	Thiếu semantic layer; text-to-SQL join sai	Query-level assertion — slide sau
Hallucinate sau khi đổi embedding	Trộn 2 vector space không tương thích	Embedding-version drift check
Yêu cầu xóa dữ liệu, agent vẫn nhớ	Soft-delete không xóa bytes gốc	Chưa có tool — bài toán mở, slide sau
Monetization model tụt hiệu quả	Ingest lỗi từ một nguồn lớn	DQ check ở biên ingestion (§15)
Model định giá lỗi nặng	Distribution shift trong training data	Drift monitoring (§10, §15)

Ngoặc cuối cột ba = nơi đào sâu hơn, không phải trích dẫn số liệu. Mẫu silent-vs-loud (frame mở đầu) tái xuất hiện độc lập ở benchmark dbt, 2 slide sau.

Agent Ngày 8 của bạn trả lời sai. Không có exception, không có log đỏ. Bắt đầu từ output và lăn ngược:

1. **Output** — agent trả lời gì, và tự tin ở mức nào?
2. **Retrieval** — chunk nào thực sự được đưa vào context cho câu trả lời này?
3. **Corpus** — chunk đó có mâu thuẫn với chunk khác, hay đã bị một bản mới hơn thay thế?
4. **Pipeline** — job ingest chunk đó lần cuối chạy khi nào, và có fail âm thầm không?
5. **Source** — hệ thống nguồn (CMS, wiki nội bộ, database...) có đang chứa data đúng, hay chính source đã sai từ đầu?

Vì sao phải lăn ngược thủ công

Không tầng nào trong 5 tầng trên tự báo lỗi. Đây chính là lý do observability (§9–§10) phải instrument **cả 5 tầng**, không chỉ dừng ở tầng cuối.

Lưu ý: Cách kể phổ biến – và **SAI**: “graph edges / trace bị khai thác qua embedding inversion.” Đồ thị **KHÔNG** phải attack surface.

Cơ chế thật (arXiv 2606.18497, Chakrabortii et al., nộp 2026-06-16):

- Xóa một vector thường chỉ **soft-delete**: record vẫn **chiếm nguyên dung lượng đĩa**
- Giá trị **float32 thô vẫn nằm nguyên** trong file index trên đĩa (vd. `index.bin`)
- Trích thẳng bằng `hnswlib.get_items()` hoặc `index.reconstruct_n()` — **bypass hoàn toàn API** của vector database
- Model inversion sau đó áp lên chính các vector thô phục hồi được từ đĩa

Bài học

Các **byte chưa bị xóa** trên đĩa mới là attack surface — không phải cấu trúc đồ thị HNSW mà nhiều bài viết vẫn quy kết.

Kiểm nghiệm trên hệ thống thật

Inversion phục hồi **25,5%** tên người từ dữ liệu tiểu sử; gần như hoàn hảo trên bản ghi y tế có cấu trúc.

Test thật: **ChromaDB, FAISS, Weaviate.**

Pinecone, Qdrant **chỉ được nhắc tới** dùng pattern soft-delete tương tự — **chưa được test**. Đừng liệt kê như đã chứng minh.

Đề xuất của tác giả

Epoch Key Rotation: AES-256-CTR, huỷ key ngay khi xoá (~0,005ms/record), kèm **deletion proof** ký ECDSA.

Bác bỏ tường minh differential privacy và orthogonal-transformation — không hiệu quả ở tầng lưu trữ.

GDPR Article 17: “*erasure must be verifiable and irreversible.*” Soft deletion không đủ.

Câu hỏi cho bạn

“Một lệnh **DELETE** trả về thành công **KHÔNG** có nghĩa là dữ liệu đã biến mất.” Nếu một user yêu cầu knowledge base Ngày 8 của bạn **quên họ** — bạn có thực sự tìm và xoá được mọi embedding phái sinh từ dữ liệu của họ không, và trong bao lâu? *Dừng 2 phút — thảo luận theo bàn.*

Dataset “ACME Insurance” (Juan Sequeda & cộng sự, data.world): 11 câu hỏi, 20 lần chạy/câu, schema 15 bảng. Open-sourced, tái lập được (dbt-labs/dbt-llm-sl-bench).

Model	Raw text-to-SQL	Semantic-layer-grounded
Claude Sonnet 4.6	90,0%	98,2%
GPT-5.3 Codex	84,1%	100,0%

dbt Labs, “Semantic Layer vs. Text-to-SQL: 2026 Benchmark Update” (dbt Developer Blog, 2026-04-07) — benchmark do chính vendor viết. Caveat của chính tác giả: baseline text-to-SQL nạp toàn bộ schema vào context, tự nhận “isn’t practical for larger datasets” – baseline yếu, có thể thiên vị hướng semantic layer.

dbt Labs (2026-04-07)

Benchmark định lượng, tự làm, baseline yếu – nhưng **open-sourced, tái lập được**.

Cube (2026-06-03)

Cùng luận điểm (“same question, same number”) nhưng **không một con số nào** – chỉ 1 quote khách hàng (Brex), không kèm dữ liệu.

Điểm định tính bền hơn con số

Semantic-layer failure **tường minh** (query báo lỗi); raw text-to-SQL failure **im lặng** (một con số trông hợp lý). Đúng mẫu “silent vs loud” – xuất hiện độc lập, không cần con số nào để đúng.

Mạch tiếp theo

Silent failure quay lại ở §9, §10, §12. Mỗi triệu chứng agent mới: tín hiệu nào bắt được nó – hay đây lại một hàng chưa có lời giải trong bảng đầu section?

Agents Schema (Fivetran+dbt, công bố cùng ngày hoàn tất merger 2026-06-01): chuẩn open-source lưu metric definition, semantic model, dbt lineage & business docs trong bảng SQL thuần — schema chung của warehouse làm context cho AI agent; hạ tầng semantic layer mà agent text-to-SQL Ngày 8 đứng trên.

Data Observability

Phát hiện data hỏng trước khi agent bị ảnh hưởng

09

Cụm từ **data observability** và bộ khung **5 Pillars** không phải chuẩn ISO hay academic taxonomy. Nguyên văn Barr Moses (đồng sáng lập **Monte Carlo**): “*I coined the term data observability in 2019*” (montecarlo.ai, cập nhật 2025-07-31).

#	Pillar	Đo cái gì
1	Freshness	Dữ liệu có được cập nhật đúng nhịp kỳ vọng không
2	Quality	% giá trị NULL, % giá trị unique, có nằm trong khoảng chấp nhận được không
3	Volume	Row count có đúng completeness kỳ vọng không
4	Schema	Cấu trúc table/column có đổi ngoài kế hoạch không
5	Lineage	Truy vết phụ thuộc upstream/downstream (chi tiết ở §11)

montecarlo.ai, “Updated Jul 31 2025”

Lưu ý: Công ty đặt ra khái niệm cũng là vendor bán tool đo nó — vài slide nữa Monte Carlo xuất hiện lại trong danh sách tool. Đây là bài học **đọc framework kèm nguồn gốc**, không phải phê phán vendor.

5 pillar, áp cho corpus RAG

- **Freshness SLI** — corpus nạp cách đây bao lâu
- **Row-count delta** — số document nạp tuần này so kỳ vọng
- **Null-rate per column** — % field metadata trống mỗi document
- **Schema drift detection** — field mới/mất/đổi type trong schema

Metric agent-era, ngoài 5 pillar gốc

- **Embedding coverage %** — % document đã có embedding hợp lệ
- **Index version consistency** — % chunk embed bởi model hiện hành
- **Retrieval staleness** — tuổi max của chunk retrieve trong 24h
- **Contradiction rate** — số cặp document mâu thuẫn trong corpus

Vì sao quan trọng

Ngày 8 xây RAG trên giả định corpus “tốt”. Đây là cách biết giả định đó còn đúng hay đã âm thầm sai.

- Cơ chế chuẩn: định nghĩa SLI latency end-to-end trên artifact then chốt, đặt SLO kèm **error budget** tường minh, map lineage từ source đến artifact, gắn check lateness/volume dọc đường

3 failure mode phải hiểu cơ chế, không chỉ tên gọi

- (a) Check “không update trong N giờ” **giả định nhịp arrival đều** — false positive khi có khoảng lặng hợp lệ (không có batch job cuối tuần), false negative khi nguồn âm thầm đổi hourly → daily mà không ai sửa threshold
- (b) **Freshness một mình không validate correctness** — table có thể update đúng lịch trong khi nội dung hoàn toàn sai (“on time but empty/garbage”)
- (c) Clock-skew/timezone giữa hệ check và hệ nguồn tạo báo động giả — đó là **lỗi đo**, không phải staleness thật

Lưu ý: Freshness pillar trả lời “có cập nhật đúng giờ không”. Nó không trả lời “nội dung có đúng không” — đó là việc của pillar Quality. Với agent corpus, table cập nhật đúng giờ nhưng rỗng vẫn khiến agent trả lời tự tin và sai.

Tool	Trạng thái 2026
Monte Carlo	Domain đổi montecarlodata.com → montecarlo.ai ; tagline mới “Trust your agents in production” — chính vendor đặt ra 5 Pillars ở slide trước (montecarlo.ai)
Elementary	Sống bên trong dbt project (dbt-native); Elementary Cloud mở rộng thành “Data & AI Control Plane” với AI agent tự tạo test/triage (elementary-data.com)
Soda	Soda 4.0 (GA 2026-01-28) chuyển hẳn sang cú pháp Data-Contracts-first thay vì SodaCL check-by-check (soda.io)
dbt test	Lớp assertion (not_null, unique, relationships) built-in ngay trong pipeline transform bạn đã viết — không phải tool riêng phải thêm vào (dbt docs)

Nguồn ghi theo từng dòng

- RAG/agent corpus hỏng không throw exception — agent vẫn trả lời trôi chảy, chỉ là sai. Đó là **silent failure**
- 5 Pillars, 6 Dimensions, hay bộ metric agent-era ở slide trước — tất cả tồn tại để bắt buộc pipeline “khai báo” khi nó bất thường, biến một failure vốn im lặng thành một alert ồn ào, **trước khi** nó tới tay agent
- Framework nào không quan trọng bằng việc **có đo hay không** — và biết rõ framework đó đến từ đâu khi bạn trình bày nó cho người khác

Tiếp theo

§10 (anomaly detection) trả lời câu hỏi kỹ thuật: đo được rồi thì phát hiện bất thường bằng cách nào.

Phát Hiện Bất Thường

Threshold, thống kê, hay ML — và cái giá của báo động giả

10

Bigeye, Monte Carlo, Anomalo đều quảng cáo ML giảm false positive so với rule tĩnh — nhóm nghiên cứu **không tìm được** một phép đo công khai, minh bạch về phương pháp, so sánh trực tiếp rules vs ML cho data quality monitoring.

Rule-Based / Threshold

- Phủ đủ failure mode **đã biết**; bằng không với cái mới
- Threshold = **một cần gạt duy nhất**: bảo thủ → nhiều false positive, nới → nhiều false negative
- Thắng khi cần interpretability, ít lịch sử, hoặc gặp **hard business invariant**
- VD agent: “document không thể có số chunk âm” — **không bao giờ** nên dùng ML cho việc này

Statistical / ML

- Thích ứng seasonality/trend hợp lệ; tự học lại “normal”
- Không giải thích được (“anomaly score 0.87” vs “row_count > 0 failed”) — khó thuyết phục người trực on-call
- Cần đủ lịch sử để train — vô dụng lúc cold-start
- **HYBRID, không either/or**: rule cho hard-constraint, ML/statistical chồng lên cho metric liên tục có seasonality

Lưu ý: Đừng chọn tool vì slide “giảm 40% false positive” của vendor. Người bị page lúc 2 giờ sáng khi agent của bạn bắt đầu trả lời sai là bạn — không phải phòng marketing của vendor.

- **Z-score:** $(x - \text{mean}) / \text{std_dev}$, flag nếu $|z| > \text{threshold}$ (thường 3) — hoạt động tốt khi metric của agent (VD số document nạp mỗi giờ) xấp xỉ normal và stationary
- **Failure (a) — masking effect:** mean/std **không robust** với chính outlier bạn muốn bắt — một spike lớn (VD một batch nạp lỗi trùng lặp hàng loạt) thổi phồng std_dev và có thể **tự che chính mình**
- **Failure (b) — giả định stationarity:** metric có trend/seasonality (VD lưu lượng ingest cuối tuần thấp hơn) sẽ false-positive ở đỉnh/đáy mùa vụ “bình thường”
- **MAD** (Median Absolute Deviation): fix robust-statistics chuẩn cho (a) — dùng median và median-absolute-deviation thay mean/std → không bị chính outlier làm lệch; vẫn dính (b)
- **STL:** tách chuỗi (VD row-count document nạp mỗi ngày cho corpus agent) thành **trend + seasonal + residual**; bất thường = độ lệch lớn trong **residual** sau khi loại trend/seasonal
- Vì sao threshold gây thơ: phân biệt được “row count thấp vì hôm nay Chủ nhật” với “row count thấp vì pipeline hỏng” — đúng failure mode rule-based hay đâm phải
- **Failure — regime change:** vỡ khi **chính pattern mùa vụ đổi** (VD sản phẩm launch dịch vĩnh viễn tỉ lệ ngày thường/cuối tuần) — decomposition gán nhầm pattern mới vào trend/residual đến khi học lại
- Chi phí tính toán/vận hành thật so với một dòng threshold — thừa nếu metric của bạn thực sự không có mùa vụ

Nguồn: *arXiv 2310.00268; JetBrains PyCharm blog (2025-01).*

Cold-start là failure mode chung của cả hai: corpus agent mới 2 tuần không đủ lịch sử để ước lượng std_dev hay seasonality theo tuần đáng tin.

- **PSI chính là Jeffreys divergence** — tổng đối xứng KL hai chiều: $PSI(P, Q) = D_{KL}(P || Q) + D_{KL}(Q || P)$. Tính đối xứng là ưu thế so với KL một chiều cho troubleshooting: hoán đổi “current vs baseline” không đổi điểm
- **Fiddler AI** (blog thường được trích): $< 0,1$ similar / $0,1-0,2$ moderately different / $> 0,2$ develop a new model
- **Truyền thống credit-risk scorecard** (dòng Naeem Siddiqi): cut trên là $> 0,25$
- → **Hai convention phổ biến nhất bất đồng về cut trên** — đó chính là bằng chứng mạnh nhất cho việc **không có threshold phổ quát**, đừng che sự bất đồng này đi khi bạn dạy lại nó
- Giới hạn khác: **binning sensitivity, univariate only** (mù với joint shift giữa hai feature), **type-blind** (báo “đã dịch”, không nói vì sao hay có quan trọng không)

Cho agent của bạn

Dùng PSI/KL theo dõi phân phối embedding/feature đầu vào cho retrieval (Ngày 8) đã dịch bao xa so với baseline — cảnh báo chỉ nói “đã dịch”, phần “có đáng lo không” vẫn là việc của bạn.

- 4 loại thay đổi: cột **được thêm**, cột bị **đổi tên**, cột bị **xóa**, cột bị **đổi kiểu** (retyped)
- **Xóa / đổi tên**: code downstream tham chiếu cột đó lỗi ngay lập tức — **loud**, dễ bắt, dễ sửa
- **Đổi kiểu**: tùy hệ thống — có thể lỗi ngay (strict schema), có thể âm thầm cast sai giá trị
- **Thêm mới**: không ai tham chiếu tới cột đó — pipeline “chạy thành công” như mọi lần. Đây là loại **nguy hiểm nhất** vì hoàn toàn im lặng, cần detection chủ động (theo dõi diff schema theo thời gian), không thể chờ code báo lỗi
- Ngay cả rule cứng cũng có góc mù kiểu này: check “expect đúng 5 giá trị status” chỉ bắt được khi giá trị **thứ 6** vi phạm một assertion đã viết sẵn — một **cột hoàn toàn mới** thì chưa ai viết assertion cho nó, nên không rule nào bắt được

Lưu ý: Với agent: một cột metadata mới lặng lẽ trôi vào corpus (VD field nội bộ không nên vào index) không gây lỗi nào — agent vẫn trả lời trôi chảy, chỉ là có thể lộ ra thứ lẽ ra không nên có trong câu trả lời. Thống kê drift sâu hơn: Ngày 27.

- Một detector **không ai tin** còn tệ hơn không có detector — nó huấn luyện đội ngũ bỏ qua đúng cái channel lẽ ra phải xem
- Cơ chế thất bại: threshold quá nhạy (rule bảo thủ, PSI cut quá thấp, z-score threshold quá chặt) tạo quá nhiều false positive → đội ngũ ngừng phản hồi → lần regression **thật** tiếp theo trôi thẳng vào agent, không ai nhìn
- Ngược lại cũng nguy hiểm: threshold quá lỏng = false negative — corpus hỏng mà không ai biết, agent trả lời sai mà không detector nào lên tiếng
- Alert fatigue được nghiên cứu nghiêm túc ở lĩnh vực lán giềng (security operations, arXiv 2605.08316, 2026-05); data quality monitoring chưa có nghiên cứu tương đương, nhưng không có lý do để tin cơ chế thất bại ở đây khác đi

Tiếp theo

§11 đào sâu lineage — trực phát hiện còn lại. §12 trả lời phần còn lại: có alert rồi thì làm gì — route, triage, runbook, ai chịu trách nhiệm. *Dừng 2 phút — thảo luận theo bàn.*

Lineage, Governance & Tuân Thủ

Data đến từ đâu, đi đâu, và luật nói gì



Lineage = bản đồ truy vấn được của *data đi qua hệ thống như thế nào* – source nào nuôi transform nào, transform nào nuôi bảng hay index nào.

- **Table-level lineage** – bảng A nuôi bảng B nuôi index C. Để tính, đủ để trả lời “job nào phụ thuộc job nào.”
- **Column-level lineage** – field cụ thể như `customers.region` chảy vào field nào ở downstream. Khó hơn nhiều (slide sau), nhưng mới thực sự trả lời “đổi cột này có làm gì với không.”

Vì sao đây không chỉ là §8 làm lại

Ở §8 bạn lần ngược 5 tầng **thủ công** khi debug một câu trả lời sai. Lineage biến việc lần ngược đó thành **truy vấn**, không phải đọc log tay. Chi tiết cơ chế thu thập + column-level: sâu hơn ở **Ngày 27**.

OpenLineage – Graduated project của **LF AI & Data** (incubation 5/2021 → graduated 7/2023). Spec/client: **v1.52.0** (2026-07-23).

Object model: Dataset • Job • Run • RunEvent • Facets.

Marquez – lineage graph UI + REST API, tự host; không phải catalog đầy đủ (glossary + DQ rule engine: cột bên).

Adoption thật, không phải marketing

Airflow maintain OpenLineage provider ngay trong repo của chính nó (2.19.0, cần Airflow $\geq$ 2.11.0)
– “adoption rộng ở tầng orchestrator”, **đừng nói** “chuẩn de facto.”

Column-level khó vì (blog **DataHub**): **ambiguous column ref** (JOIN trùng tên cột) • **SELECT *** cần schema sống để expand • **dynamic SQL/UDF/macro** lạ cần emit thủ công.

Benchmark của chính DataHub (vendor tự đo)

SELECT: **90%/85%** (table/column). CTAS: **92%/88%**. Parser **SQLGlot v30.0.3**, 30+ dialect (fork acryl-sqlglot đã retired).

Trạng thái trung thực

Đủ để triage impact analysis; **KHÔNG** đủ để tự động hoá cascade xoá GDPR không người kiểm. Cơ chế thu thập, object model, độ chính xác parse SQL: sâu hơn ở **Ngày 27**.

Blast radius – heuristic **không chính thức** của giới hành nghề, không phải định nghĩa chuẩn hoá: *tổng bề mặt downstream bị ảnh hưởng khi một nguồn hỏng*.

Pattern vận hành đáng học

Kết hợp **lineage graph + query history** → biến một escalation mơ hồ thành điểm tác động nghiệp vụ **đo được và xếp hạng ưu tiên được**. Đây không phải checkbox tuân thủ – đây là cơ chế **triage incident** theo chi phí.

Minh hoạ trên corpus nuôi agent:

Thay đổi ở nguồn	Downstream vỡ	Agent trả lời sai gì
customers.region đổi kiểu string → enum	Materialized view revenue-by-region vỡ join	“Khu vực nào doanh thu cao nhất quý này?” – sai, im lặng
products.description bị truncate ở nguồn	Chunk embedding sinh từ description cũ, chưa re-embed	Agent trích mô tả sản phẩm lỗi thời, vẫn tự tin

Catalog	Vai trò	Trạng thái
Apache Polaris	Table catalog (Iceberg, nay thêm Delta Lake)	ASF Top-Level Project (2026-02-19); v1.6.0 (2026-07-08)
Unity Catalog OSS	Table catalog, Databricks hiển tặng LF AI & Data	LF AI & Data SANDBOX (Apache 2.0); v0.5.1 (7/2026)
DataHub	Metadata/discovery catalog	“3.000+ tổ chức” là con số bản OPEN SOURCE , không phải khách trả tiền
OpenMetadata (Collate)	Metadata/discovery catalog	v1.13.3 (2026-07-31), ship MCP tool support để agent query metadata được governed
Amundsen (Lyft)	Metadata/discovery catalog, đời đầu	LF AI & Data INCUBATION ; nhịp phát triển chậm rõ rệt so với hai cái trên

Polaris/Unity Catalog nằm SÁT table format, bên dưới lakehouse; DataHub/OpenMetadata/Amundsen crawl mọi thứ bên trên, thêm business glossary + lineage UI. Sâu hơn ở Ngày 27.

Tầng	Luật / Nghị định	Hiệu lực	Phạm vi chính
1	Luật An ninh mạng + Nghị định 147	(đang áp dụng)	Localization dịch vụ internet
2	Luật Dữ liệu số 60/2024/QH15 (thông qua 2024-11-30)	2025-07-01	Toàn bộ dữ liệu số (không chỉ dữ liệu cá nhân), hiệu lực ngoài lãnh thổ; phân loại “important”/“core” data
3	PDPL, Luật 91/2025/QH15 (thông qua 2025-06-26) + Nghị định 356/2025/NĐ-CP (2025-12-31)	2026-01-01	Thay Nghị định 13/2023/NĐ-CP; consent-centric, dữ liệu cá nhân

*Nghị định hướng dẫn PDPL KHÔNG còn đang chờ – 356/2025/NĐ-CP đã ban hành cùng thời điểm, khung đã hoàn chỉnh.
Áp dụng cho pipeline nuôi RAG/agent Ngày 8 y hết pipeline BI.*

- **Core data** ($\geq 1.000.000$ công dân): chấp thuận văn bản **TRƯỚC** từ Bộ Công an hoặc Bộ Quốc phòng trước khi chuyển ra nước ngoài (Luật Dữ liệu 60/2024).
- **Important data** (100.000–999.999 công dân): nộp hồ sơ đánh giá tác động **15 ngày TRƯỚC KHI chuyển** – notification, không phải pre-approval. (Chiều ngược lại đang lưu hành là **sai**.)
- **PDPL**: consent phải “rõ ràng, cụ thể”; carve-out “legitimate interest” **hẹp hơn** GDPR; rút đồng ý bất cứ lúc nào, **không hồi tố**.
- Chuyển dữ liệu cá nhân xuyên biên giới: **Transfer Impact Assessment (TIA)** nộp Bộ Công an (trừ vài miễn trừ: cơ quan nhà nước, dữ liệu nhân viên qua cloud, khẩn cấp).
- Chế tài: chuyển xuyên biên giới trái phép **tới 5% doanh thu năm liền trước**; mua bán dữ liệu cá nhân trái phép **tới 10 lần doanh thu**; hình sự tới 7 năm tù.

Ràng buộc trên chính kiến trúc bạn đã dựng

Pipeline đẩy dữ liệu người dùng ra khỏi Việt Nam – ví dụ tới vector DB hosted ở Mỹ hoặc LLM API nước ngoài – cần một **TIA** (PDPL) VÀ, nếu dữ liệu thuộc loại “important”, một hồ sơ nộp trước 15 ngày cho Bộ Công an/Bộ Quốc phòng (Luật Dữ liệu). Đây là ràng buộc trên chính kiến trúc RAG bạn đã dựng ở **Ngày 8**.

Lưu ý: Article 10 (data governance cho hệ thống high-risk) **KHÔNG** có hiệu lực 2026-08-02. AI Omnibus thông qua 2025-11-19, thoả thuận chính trị 2026-05-07, **CÓ HIỆU LỰC 2026-07-27** – dời lịch high-risk sang lịch so le.

Nghĩa vụ	Hiệu lực
Prohibited practices	2025-02-02 (đã áp dụng)
Article 53 – GPAI training-data summary	2025-08-01 (đã áp dụng)
Article 50 – đánh dấu nội dung AI (machine-readable)	2026-08-02 – KHÔNG bị hoãn
Article 10 – data governance cho high-risk	2027-12-02 (đã dời)
AI nhúng trong sản phẩm quản lý (thang máy, đồ chơi...)	2028-08-02

Bẫy stale-source – và vì sao Article 50 chậm agent của bạn

Tracker bên thứ ba `artificialintelligenceact.eu` **cập nhật lần cuối 2024-08-01**, vẫn hiển thị lịch tiền-Omnibus – luôn kiểm ngày cập nhật nguồn trước khi trích. Nếu agent bạn dựng tạo nội dung phục vụ người dùng ở EU, **Article 50** (hiệu lực 2026-08-02, tuần này) yêu cầu đánh dấu nội dung đó machine-readable.

EDPB Coordinated Enforcement Action lần 4 (CEF 2025), về Article 17: **764** controller trả lời, **32** DPA tham gia; **9** mở/tiếp tục điều tra chính thức, **23** fact-finding.

7 thách thức EDPB liệt kê:

1. Quy trình nội bộ xử lý yêu cầu không đầy đủ
2. Thông tin cung cấp cho cá nhân không đủ
3. **Dựa vào anonymisation kém hiệu quả thay vì xoá thật**
4. Thực hành không nhất quán
5. Khó xác định thời hạn lưu trữ
6. **Khó xoá dữ liệu khỏi hệ thống BACKUP**
7. Khó cân bằng quyền xoá với các quyền khác

Liên quan trực tiếp tới pipeline bạn xây

#3 và #6 là của bạn. Trên vector store, xem **§8 (Ghost Vectors)** – bằng chứng kỹ thuật vững nhất cho vì sao một lệnh DELETE trả về thành công **không có nghĩa** dữ liệu đã biến mất khỏi agent.

Alerting, SLA & Incident Run-book

Khi pipeline hỏng lúc 2 giờ sáng thì chuyện gì xảy ra

12

Nguyên nhân gốc data incident	Tỉ lệ
Pipeline execution faults	26,2%
Real-world variation	20,0%
Ingestion disruptions	16,6%
Platform instability	15,2%
Intentional changes	14,2%
Schema drift	7,8%

~1 vấn đề chất lượng dữ liệu trên mỗi 10 bảng mỗi năm (Monte Carlo, 2026-06). **Pipeline execution fault** — nguyên nhân lớn nhất — chính là kiểu lỗi “job ingest fail âm thầm” đã gặp ở §8/§9: không alert đúng chỗ thì agent cứ trả lời bằng corpus cũ mà không ai biết. Không có ước lượng “chi phí bad data” đủ tin cậy để trích — bằng chứng tốt nhất hiện có vẫn là bảng phân rã này, không phải một con số tổng.

Monte Carlo, “Data Quality Statistics & Insights From Monitoring +11 Million Tables in 2026” (2026-06) — telemetry vendor tự báo trên hơn 11 triệu bảng, chưa kiểm toán độc lập, nhưng là loại bằng chứng khác và mạnh hơn khảo sát cảm nhận (survey) cũ.

Ba cam kết cụ thể về một dataset:

- **Freshness** — dữ liệu mới tới mức nào (vd: corpus HR cập nhật $\leq 24h$) — agent đang trả lời dựa trên version nào tại thời điểm query?
- **Completeness** — có thiếu record/tài liệu nào không (vd: 100% chính sách hiện hành phải có mặt trong corpus)
- **Availability** — index/query endpoint có sẵn sàng phục vụ agent không — khác uptime của service API

Cam kết với ai? Không phải với end-user cuối (họ không biết SLA tồn tại) — mà với đội sở hữu sản phẩm agent, đội vận hành pipeline, đôi khi cả pháp lý/compliance.

*(Ngày 13 sở hữu SLO của service — latency/uptime API.
Đây là SLA của chính bộ dữ liệu nuôi agent.)*

Error budget, không chỉ ngưỡng tĩnh

Ngưỡng tĩnh không đủ: cơ chế đầy đủ + 2 failure mode của freshness đã ở §9. Ở đây chỉ cần nhớ: SLO phải có error budget, không phải một con số N giờ.

Mức độ nghiêm trọng:

- **Page** (đánh thức người trực) — SLA cứng bị vi phạm VÀ đang ảnh hưởng câu trả lời agent ngay lúc này: corpus production ngừng cập nhật, gate fail-closed chặn toàn bộ index
- **Ticket** — vấn đề đã bị gate chặn trước khi tới agent (§7), hoặc mới chớm ngưỡng cảnh báo — xử lý trong giờ hành chính
- **Dashboard entry** — tín hiệu thông tin/drift nhẹ, chưa cần hành động — review định kỳ

Routing theo ownership:

- Alert phải tới đúng team sở hữu dataset (ownership đã định nghĩa trong data contract, §7) — không mặc định đổ về đội vận hành agent nếu lỗi nằm ở nguồn upstream
- Nếu agent team tự vận hành pipeline riêng (RAG nội bộ), alert route thẳng về họ

Lưu ý: Ngưỡng càng nhạy càng nhiều false positive → alert fatigue — cơ chế rules-vs-ML đã phân tích ở (§10). Alert sai severity sẽ bị lờ, đúng lúc bạn cần nó nhất.

1. **Detect** — tín hiệu nào vừa fire, từ đâu (§10)
2. **Isolate** — dataset nào bị ảnh hưởng? downstream consumer nào đang đọc nó? cụ thể **agent/tool** nào đang trả lời bằng data này?
3. **Mitigate** — quyết định ngay: fail closed, serve stale, hay tắt hẳn tool đang dùng data này? (chi tiết slide sau)
4. **Fix** — sửa root cause đúng tầng — pipeline, source, hay chính bản ghi dữ liệu
5. **Verify** — chạy lại quality gate (§7) trước khi mở lại — đừng tin bằng mắt
6. **Post-mortem** — ghi lại nguyên nhân và hành động (thường bị bỏ qua — slide sau)

Vì sao cần cả 6 bước

Runbook là cách một đội biến failure vốn **IM LẶNG** (§8) thành một phản ứng **ỒN Ì**, có người chịu trách nhiệm — ngược hoàn toàn với cách nó bắt đầu.

Lựa chọn	Agent làm gì	Chọn khi
Serve stale-nhưng-đúng	Trả lời bằng version corpus cũ, không báo user	Data cũ vẫn đúng và ít rủi ro hơn dùng data mới đáng ngờ; gate đã fail-closed từ §7
Serve kèm cảnh báo staleness	Trả lời bình thường, thêm dòng “thông tin có thể lỗi thời kể từ [ngày]”	User cần tự đánh giá rủi ro — vd domain pháp lý, tài chính
Tắt hẳn tool/agent liên quan	Từ chối trả lời hoặc route sang người — không dùng data đáng ngờ	Rủi ro trả lời sai cao hơn rủi ro không trả lời — vd y tế, an toàn

“Không tồn tại thứ tương đương văn hoá SRE post-mortem cho data incident; phần lớn thất bại data quality chỉ được công bố khi chúng chạm vào một báo cáo tài chính hoặc một cơ quan nhà nước.” — Tổng hợp từ tài liệu ngành, 2026 — chi tiết các case cụ thể ở §15, không nhắc lại ở đây.

Hệ quả cho bạn

Không có chuẩn ngành làm sẵn bước 6. Bạn phải **tự lấp khoảng trống này** bằng chính observability practice của mình — nổi thẳng sang paging/on-call practice ở **Ngày 13**. Một post-mortem tối thiểu: nguyên nhân gốc, dataset/agent bị ảnh hưởng, thời gian phát hiện—sửa, và một thay đổi cụ thể để lần sau phát hiện sớm hơn.

Bối cảnh: 2 giờ sáng, alert freshness breach fire trên corpus “chính sách nhân sự” — nguồn nuôi agent HR nội bộ. Job ingest đã fail âm thầm 3 ngày liên tiếp; không ai để ý vì threshold cũ đặt ngưỡng 7 ngày.

- ☐ Agent HR vẫn đang trả lời — không exception nào được ném ra
- ☐ Trong 3 ngày đó, một chính sách nghỉ phép đã được cập nhật ở hệ nguồn nhưng chưa vào corpus
- ☐ Không rõ có nhân viên nào đã hỏi agent về chính sách này chưa

Yêu cầu: chạy qua đủ 6 bước runbook cho tình huống này — ở bước Mitigate, chọn 1 trong 3 lựa chọn (slide trước) và giải thích vì sao; viết 3 dòng post-mortem tối thiểu (bước 6) dù biết văn hoá này không có sẵn.

Orchestration & Automation

Chạy pipeline tự động, chạy lại an toàn

13

Orchestrator — Hệ thống định nghĩa pipeline như một **DAG** (đồ thị phụ thuộc không chu trình) gồm các **task**; nó quyết định task nào chạy khi nào, theo dõi trạng thái, và chạy lại khi fail.

Airflow 3.0 (GA 2025-04-22) đổi kiến trúc, không chỉ đổi UI

- **DAG Versioning** — một DAG run gắn với đúng phiên bản code đang active lúc bắt đầu; sửa file DAG giữa chừng không làm hỏng run đang bay
- **Task Execution API** — worker không còn truy cập DB Airflow trực tiếp, chạy task qua một API có tài liệu → task isolation, thực thi đa ngôn ngữ/từ xa
- **Assets** thay **Datasets** (2.4) — phản ứng được với sự kiện bên ngoài, không chỉ khi một DAG khác chạy xong
- **3.1.0** (2025-09-25) thêm Human-in-the-Loop task, Deadline Alerts, và “Inference Execution” — DAG gọi được như một synchronous call

Lưu ý: Airflow 2.x đã EOL ngày 22/4/2026 — đã qua, không phải sắp tới. Limited Maintenance từ 2025-10-22 tại 2.11.2. Stable hiện tại: **3.3.0**. Migrate vỡ: SubDAG bị bỏ, `execution_date` bị bỏ, provider import path đổi. Ngay cả GCP Composer 3 (managed) vẫn còn ship image Airflow 2.x tính đến 7/2026 — đi **sau** upstream, không song song với nó.

Orchestrator 2026 không chỉ chạy BI batch: theo khảo sát của **Astronomer** (đơn vị bán managed Airflow) trên toàn bộ người trả lời, **32%** báo cáo có use case GenAI/MLOps, tăng lên **62%** riêng ở nhóm khách hàng trả tiền (Astronomer, “State of Airflow 2026”). Pipeline nạp corpus RAG của bạn (Ngày 8) chạy trên đúng loại hạ tầng này.

Tool	Hợp việc gì	Trạng thái 2026
Airflow 3.x	Batch ETL/ELT truyền thống, install base lớn nhất	Stable 3.3.0 ; GCP Composer 3 ship cả 3.x lẫn 2.x (2026-07-29)
Prefect [†] Dagster [†] Temporal	Dynamic Python, team ML/platform Asset-centric, partitioning/cataloging Agent/service reliability (Ngày 9) — không phải drop-in thay Airflow	Cùng công ty với Dagster từ 2026-07-13 Asset model ổn định từ ~2022 “Crash-proof execution”: state sống sót qua crash
Kestra	YAML-first, config-as-workflow	1.3 LTS (2026-03-03); Assets từ 1.2 (2026-01-13)
Flyte 2.0	Union: “durable AI runtime” — Python control flow + resource-aware retry	GA 2026-08-04
Netflix Maestro	Batch quy mô lớn, hỗ trợ cyclic workflow	OSS 2024-07-22 (Apache-2.0); chỉ 167 commit (community nhỏ)

Nguồn ghi theo từng dòng

[†] **Cùng một công ty từ 2026-07-13.** Prefect công bố mua lại Dagster Labs — xác nhận chéo trên 3 domain độc lập; giá không công bố; ~40 nhân sự Dagster chuyển sang. Nick Schrock rời công ty hợp nhất, lý do ông nêu: hai sản phẩm đã thành “*complementary products rather than direct competitors*”. Dagster OSS tiếp tục phát triển tích cực; Dagster+ giữ thương hiệu riêng. Câu hỏi đúng không còn là “ai thắng” mà: **bạn cần dynamic Python hay asset-centric?**

- ✓ **Cron** — lịch cố định (vd 2 giờ sáng mỗi ngày); đơn giản nhưng chạy dù data có về kịp hay không
- ✓ **Event-driven** — kích hoạt khi có sự kiện ngoài (vd message SQS); Airflow Assets (3.0) hỗ trợ trực tiếp
- ✓ **Dependency/asset-driven** — schedule khi một asset upstream vừa cập nhật, không chỉ khi DAG khác chạy xong (trang sau)
- ✓ **Backfill** — chạy lại cho một khoảng thời gian quá khứ, thường sau khi sửa transform logic hoặc đổi embedding model (trang sau)

Cho corpus RAG của bạn

Một pipeline nạp corpus thực tế thường cần cả 4: cron re-embed ban đêm, event-driven khi có document mới, backfill khi đổi embedding model.

Declarative / Asset-Centric

- **Airflow Assets** — Datasets (2.4) nâng thành primitive hạng nhất ở 3.0
- **Kestra** — YAML-first từ đầu; Assets thêm ở 1.2 (2026-01-13)
- **Dagster** — asset là đơn vị scheduling, **phổ biến hoá** từ ~2022, không phải “phát minh”

Ý chính: schedule theo *data đã về*, không theo đồng hồ (diễn giải của giảng viên).

Dynamic Python / Escape Static DAG

- **Prefect** — native Python control flow (if/else, while) chạy trực tiếp từ bản **2.0**, không phải riêng 3.x
- **Flyte 2.0** (GA 2026-08-04) — đảo ngược model static-compile của Flyte 1.x: task gọi task, loop/try-except native

Corpus RAG: asset-driven khi nhiều nguồn phụ thuộc nhau qua dữ liệu; dynamic Python khi logic rẽ nhánh phức tạp hơn DAG tĩnh.

Backfill: mặc định tốn hơn bạn nghĩ

- Dagster mặc định launch **một run riêng cho mỗi partition** khi backfill — lãng phí overhead khi compute bên dưới (Spark/Snowflake) đã tự parallelize (Dagster docs)
- Khắc phục: `backfill_policy = single_run` — asset code nhận nguyên **một dải partition** thay vì từng partition key một

Khoảng trống watermark — orchestrator batch không có

- (Cùng từ, nghĩa hẹp hơn ở §3 — ingestion cursor; đây là nghĩa streaming-window của Beam.)
Watermark (Apache Beam docs): *“a guess as to when all data in a certain window is expected to have arrived”* — dữ liệu tới sau khi watermark qua window đó là **late data**
- Airflow/Dagster/Prefect **không có khái niệm watermark/late-arriving-data native** như hệ streaming (Beam/Flink) — đây là khoảng trống thật, không phải thiếu tài liệu

Lưu ý: Backfill lại corpus RAG sau khi đổi chunk size hoặc embedding model chính là lúc idempotency (trang sau) quan trọng nhất: chạy lại hàng nghìn partition, và nếu có document “tới trễ” so với watermark tưởng tượng, orchestrator của bạn sẽ không tự biết.

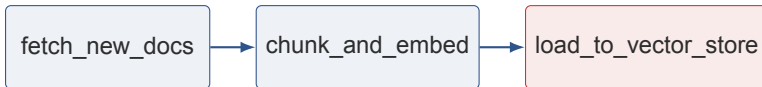
- **Retry với exponential backoff** — task fail được thử lại sau khoảng chờ tăng dần, tránh đội thêm tải vào một hệ thống nguồn đang gặp sự cố
- **Dead-letter queue (DLQ)** — record fail sau khi hết số lần retry cho phép không bị vứt bỏ, mà đưa qua hàng đợi riêng để xử lý/debug thủ công sau
- **Partial failure** — một DAG run có thể có vài task fail trong khi nhánh khác vẫn thành công; orchestrator phải quyết định task downstream nào bị chặn, task nào độc lập vẫn chạy tiếp
- Airflow best-practices docs: *“You should treat tasks in Airflow equivalent to transactions in a database... you should never produce incomplete results from your tasks.”*
- *“Do not use INSERT during a task re-run, an INSERT statement might lead to duplicate rows in your database. Replace it with UPSERT.”*
- *“Read and write in a specific partition. Never read the latest available data in a task... use data_interval_start as a partition.”* Không dùng `datetime.now()` trong task cho tính toán

Lưu ý: Đừng gọi orchestrator batch nào là “exactly-once”. Tốt nhất chúng cho **at-least-once + idempotent consumer = “effectively-once”**. Exactly-once thật thuộc về stream-processing engine (Kafka Streams/Flink), không phải orchestrator.

Với vector store: retry/backfill trên task không idempotent **chèn thêm bản sao chunk** — agent trích dẫn trùng và tin tưởng sai lệch.

Luật chơi

Nhìn DAG dưới đây. Task cuối tự động retry **3 lần** khi timeout. Xác định: nó sai ở đâu, và điều gì xảy ra với vector store nếu bạn không sửa? Không có đáp án trên slide — thảo luận theo bàn.



```
-- inside load_to_vector_store:  
INSERT INTO doc_chunks (id, content, embedding) VALUES (...)
```

Chi Phí & Kinh Tế Pipeline

Cái gì thực sự tốn tiền khi bạn nuôi một agent bằng data

14

- ☒ **Ingestion & egress** — trả bằng thông khi dữ liệu rời cloud vendor này sang nơi khác
- ☒ **Storage** — warehouse (Bronze/Silver/Gold, §5) cộng vector index, tính theo GB/tháng
- ☒ **Compute cho transform** — mỗi lần chạy dbt model / rescan bảng để build Gold layer là một hoá đơn query, không phải chi phí một lần
- ☒ **Embedding API calls** — mỗi document, mỗi lần chunk lại, mỗi lần đổi model là một cuộc gọi API tính phí theo token
- ☒ **Vector index storage & rebuild** — lưu vector rẻ, nhưng **build lại** HNSW sau khi đổi model không miễn phí CPU/RAM

Agent Day 8–9 của bạn không chạy miễn phí sau demo. **Ba dòng cuối là ba dòng tăng theo mỗi lần bạn thử nghiệm** — đổi chunking, đổi model, re-index để test lại.

Provider	Model	\$/1M token	Ghi chú
OpenAI	text-embedding-3-small	\$0,02	Batch không công bố — đừng trích "\$0,01" như sổ vendor
OpenAI	text-embedding-3-large	\$0,13	cùng caveat batch
Google	gemini-embedding-001	\$0,15 (batch \$0,075)	Trang tự đề 2026-07-30
Voyage AI	voyage-4-lite / voyage-4	\$0,02 / \$0,06	200M token free cho account mới; thuộc MongoDB từ 2025
Cohere	Embed 4	hết bán theo token	Chỉ dedicated: Small \$4/giờ hoặc \$2.500/tháng

Giá fetch 2026-08-05, developers.openai.com; ai.google.dev; docs.voyageai.com; cohere.com/pricing — **kiểm tra lại vào đúng ngày dạy**

Giả định: 500.000 document $\times$ ~800 token = **400 triệu token**.

Hoá đơn: 3-small **\$8,00** · 3-large **\$52,00** · gemini-embedding-001 **\$60,00** · voyage-4 \$24,00 → **\$12,00** lần đầu (200M free).

Lưu vector thì rẻ: $500.000 \times 1.536 \times 4 \text{ byte} \approx \mathbf{3,07 \text{ GB}}$; kể cả overhead 2–4 $\times$ cho HNSW vẫn dưới 15GB — sai số làm tròn so với hoá đơn embedding. **Tiền không nằm ở chỗ bạn nghĩ.**

- Vector space từ hai embedding model khác nhau **không tương thích** — và **không provider nào bán giá incremental-diff** cho việc đổi model
- Kết quả: một quyết định “thử model embedding mới xem có tốt hơn không” tốn **toàn bộ hoá đơn embedding lần nữa** — \$8–\$60 trong ví dụ đồ chơi ở trên, nhân $100\text{--}1000\times$ cho corpus enterprise thật
- **Cộng thêm** chi phí rebuild index trong vector DB — build lại HNSW graph không miễn phí CPU/RAM, đây là chi phí compute ẩn ít người tính vào hoá đơn
- Ngoại lệ đáng lưu ý: 200M token miễn phí của Voyage phủ nửa lần chạy 400M đầu tiên — điều này thay đổi đáng kể câu chuyện “đổi model = trả full giá lại” **cho riêng provider đó**

Khi nào thực sự phải trả giá này?

Quyết định kỹ thuật “re-embed toàn bộ hay upsert là đủ” — và một kỹ thuật thử nghiệm để né chi phí này (Drift-Adapter) — đã dạy ở §5. Ở đây: mỗi lần đội bạn nói “thử model embedding khác xem”, đó là một dòng ngân sách, không phải thử nghiệm miễn phí.

Hạng mục	Giá	Ghi chú
BigQuery query	\$6,25/TiB scanned	1 TiB đầu/tháng free; giá region US
AWS Athena	\$5,00/TB scanned	Trang giá chính thức
S3 egress internet	\$0,09/GB	Chỉ xác nhận ví dụ Ireland — không rõ tier free cho US

Ví dụ: RAG service rescan 2

TiB warehouse/đêm cho re-index, cộng 50GB/tháng chunk payload qua internet:

- Rescan/đêm: $2 \text{ TiB} \times 30 = 60 \text{ TiB} - 1 \text{ TiB free} \Rightarrow 59 \text{ TiB} \times \$6,25 \approx \textbf{\$368,75/tháng}$
- Egress 50GB $\times \$0,09 = \textbf{\$4,50/tháng}$

Bài học: compute-over-data (scan lặp lại) là đòn bẩy chi phí chi phối, không phải storage/egress — lập luận định lượng cho **incremental ingestion/CDC/partitioning** (§3–§4).

BigQuery pricing (2026-08); aws.amazon.com/athena/pricing — giá theo region, egress chỉ xác nhận ở ví dụ Europe/Ireland

- **Không tồn tại benchmark chi phí đa-vendor được audit độc lập nào mới.** Nghiên cứu chặt chẽ nhất còn methodology công bố vẫn là **Fivetran, 2022-12-19** (Redshift, Snowflake, BigQuery, Databricks, Azure Synapse; TPC-DS 99 query, 1TB) — kết luận: các warehouse “đều có tốc độ xuất sắc”, runtime cụm quanh ~\$16/giờ, **gần như ngang bằng**, chưa từng chạy lại
- **Vì sao so sánh \$/TB đa-vendor gần vô nghĩa:** Snowflake credit, Databricks DBU, BigQuery TB-scanned đo những đơn vị công việc khác nhau — “apples to oranges to grapefruit”
- **Khuyến nghị:** dùng list price của *một* vendor có nhãn rõ, hoặc dạy thẳng vấn đề “đơn vị không so sánh được” — đừng dựng slide “\$/TB” chéo vendor

Khi một vendor nói “rẻ hơn đối thủ X đến Y%” mà không kèm methodology công khai, đọc nó như **marketing**, không phải fact — loại số bạn sẽ gặp khi chọn vector DB/warehouse cho agent của mình.

Sự Có Thật & Những Con Số Đừng Dừng

Bằng chứng nào đáng tin — và bằng chứng nào chỉ được lặp lại

“...the consequences of ingesting bad data from a large customer... reduced the efficacy of such products.” — Form 10-Q, Unity Software Inc., kỳ kết thúc 2022-06-30, nộp SEC **2022-08-09** — xuất hiện HAI lần, trong Risk Factors và MD&A

- Ước tính tác động doanh thu: **\$110 triệu** trong 2022 (earnings call 2022-05-10 — verified-secondary, số này **KHÔNG** nằm trong chính văn 10-Q)
- Sự cổ kích hoạt một **securities class-action** chống Unity: bad data trở thành vấn đề luật chứng khoán, không chỉ vấn đề kỹ thuật

Lưu ý: Cụm “đầu đọc training data” **không có trong hồ sơ** — chữ “training data” xuất hiện 0 lần. Đó là cách kể phổ biến, không phải điều Unity công bố. Chỉ trích phần Unity thực sự nói.

Đây là dòng mạnh nhất trong cả case-study set: một công ty đại chúng viết bằng văn bản pháp lý rằng data bẩn làm họ mất tiền. Nếu pipeline nạp corpus cho agent của bạn nhận thẳng dữ liệu từ một nguồn ngoài không qua validation gate (§7), đây là hậu quả có thật của con đường đó.

Zillow Offers — 8-K, 2021-11-02

\$304 triệu write-down Q3 + **\$240–265 triệu** lỗ dự kiến Q4 → **~\$550 triệu+**; cắt **~25%** nhân sự (~2.000 việc làm)

Zillow tự nói: “*unpredictability in forecasting home prices*” — **không** nói tường minh “pipeline hỏng”

Số liệu = xác nhận. Nhân quả “model định giá train trên dữ liệu cũ giữa cơn sốt nhà đất đảo chiều” = phân tích bên ngoài, hợp lý nhưng **KHÔNG** phải lời Zillow.

PHE — gov.uk, 2020-10-04

15.841 ca dương tính COVID (25/9–2/10/2020) không nạp kịp hệ truy vết, do “a technical issue... files... exceeded the maximum file size”

PHE tự phản bác: cá nhân vẫn nhận kết quả đúng; **không** có ổ dịch địa phương nào bị bỏ sót

Cơ chế “trần dòng Excel .xls” là cách kể của báo chí thời điểm đó, **KHÔNG** nằm trong tuyên bố gốc — đừng khẳng định con số 65.536 dòng như fact.

Data/UI Failure — Citi/Revlon 2020

\$893.944.008,52 chuyển nhầm (2nd Circuit, 2022-09-08)

Nguyên nhân: mặc định Flexcube gửi wire trừ khi bị chặn; người thao tác chỉ tick ô PRINCIPAL, bỏ sót FRONT+FUND; quy trình duyệt “six-eye” ba người — **cả ba đều bỏ sót**

Tòa: *VACATED* và *REMANDED* (không phải “reversed”)

Deployment Failure — Knight Capital 2012

SEC: lỗi “**over \$460 triệu**” (không phải \$440M — đó là số Knight tự báo pre-tax)

Code “Power Peg” ngừng từ 2003 bị bỏ lại production; một flag tái dùng kích hoạt lại nó; đợt deploy mới **bỏ sót một server**. Phạt SEC **\$12.000.000**

Sáp nhập vào KCG Holdings 2013-07-01 (~11 tháng sau, không phải “vài tháng”)

Citi/Revlon = **giao diện nhập liệu chính là data pipeline** → validation, gate, review nhiều mắt (§7). Knight = code chết bị deploy nhầm → **không** data quality/observability nào cứu được; hàng phòng thủ đúng là feature flag, kill switch, xóa code chết trước deploy.

Lưu ý: Pipeline agent của bạn hỏng: đây là data bắn đi vào, hay code/config cũ bị bật nhầm lên production? Hai câu hỏi, hai bộ công cụ.

Moffatt v. Air Canada, **2024 BCCRT 149** (2024-02-14): tòa xác lập trách nhiệm **negligent misrepresentation**. Bồi thường chính xác **\$812,02** (\$650,88 + \$36,14 lãi + \$125 phí) — \$880 là số Moffatt đòi, không phải một mức bồi thường khác.

“Air Canada suggests the chatbot is a separate legal entity that is responsible for its own actions. This is a remarkable submission.” — đoạn 27, Tribunal Member Christopher C. Rivers

Lưu ý: Cách kể “hai trang chính sách mâu thuẫn / stale retrieval” bị bác bỏ: chỉ có **MỘT** trang bereavement, và trang đó **đúng**. Chatbot mâu thuẫn với chính trang nó hyperlink tới. Tòa **không có phát hiện nào** về retrieval/RAG.

Framing đúng: agent của bạn mâu thuẫn với chính source of truth của bạn → công ty chịu trách nhiệm pháp lý. Đây chính xác là lỗi mà contradiction-detection và single-source-of-truth (§11) tồn tại để chặn — không phải lỗi mà “làm retrieval tươi hơn” sửa được, vì retrieval còn chưa được xác nhận là nguyên nhân.

Bản deck này từng trích **Sculley et al., NeurIPS 2015** cho “80% thời gian AI là data work”. Paper đó **không chứa số liệu thời gian nào** — nhưng **CÓ** chứa: “(at most) 5% machine learning code and (at least) 95% glue code”, một tỉ lệ **CODE**.

Lưu ý: Đừng nói “paper không có phần trăm nào” — sinh viên mở PDF sẽ thấy 95%/5% ngay. Nói đúng: **có phần trăm, nhưng đo cái khác**.

Chuỗi rửa citation (audit của Leigh Dodds, 2020-01-31): trang marketing IBM → blog IBM 2017 → HBR 2018 → khảo sát gốc **CrowdFlower 2016** — một vendor gán nhãn dữ liệu, n~80, tự chọn mẫu. Dodds gọi thẳng: “a bullshit statistic.”

CrowdFlower 2016 60%+19% (cộng riêng lẻ = 80%)

Kaggle 2018 (lớn, ~26% không vendor)

Khoảng trung thực: 15–80%, tùy định nghĩa “cleaning”.

Thay bằng gì?

Sambasivan et al., **CHI 2021** (Google Research), “Everyone Wants to Do the Model Work, Not the Data Work”: **92% prevalence** của data cascade — đo **mức phổ biến của tổn hại**, không phải giờ. Đây là phỏng vấn định tính **53 người thực hành**, **KHÔNG** phải khảo sát mức quần thể — nói rõ điều này khi trích.

- **Gartner “\$12,9 triệu/năm”** — khảo sát **tự báo cáo**, không phải chi phí được audit; mẫu tự chọn lọc về phía công ty đã tin mình có vấn đề data quality
- **IBM “\$3,1 nghìn tỷ/năm”** — từ một infographic marketing 2016; bài HBR (Redman, 2016-09-22) hay bị chuỗi vào cùng thực ra ghi **\$3 nghìn tỷ** (không phải 3,1) và **không hề gán con số đó cho IBM** — một con số thứ BA bị nổi nhảm đây
- **“793 giờ data downtime/tháng”** (Monte Carlo) — một **tích số suy ra** (61×13), không xuất hiện trong nguồn primary nào của họ
- **“\$600 triệu ARR gộp” Fivetran+dbt** — không có trong press release hoàn tất merger, không có nguồn tài chính primary nào

Điểm chung: mỗi số này được tái trích liên tục qua nhiều năm mà không ai đo lại lần nào.

- ✓ Ai xuất bản/tài trợ con số này — có bán chính giải pháp mà con số “chứng minh” là cần thiết không?
- ✓ Cỡ mẫu n là bao nhiêu? Tự chọn mẫu hay đại diện?
- ✓ Con số đo **chính xác** cái gì — % thời gian? % code? % người xếp hạng “quan trọng nhất”?
- ✓ Năm nào? Đã được đo lại, hay chỉ được tái trích suốt một thập kỷ?
- ✓ Bạn đọc được nguyên văn nguồn gốc, hay chỉ toàn thấy nó trong một blog trích một blog khác?

Áp cùng bài kiểm tra này cho benchmark vendor ở tool bạn chọn (§9, §13) và cho chính con số bạn viết trong báo cáo đánh giá agent của mình.

Frontier 2026

Cái gì thực sự mới — và cái gì mới chỉ là marketing

16

Tầng marketing (không truy được nguồn)

- Nhiều con số (adoption AI agent, % thời gian maintenance toil, số nguồn dữ liệu/công ty) lưu hành khắp blog vendor 2026 (Medium, Integrate.io, LatentView, Nitor Infotech)
- Không truy được về báo cáo gốc – kể cả số “trông giống Gartner”; đây là content marketing vendor/vendor-adjacent, **không phải nghiên cứu**
- Nhận diện: càng cụ thể, càng lưu hành rộng, không kèm link gốc – càng đáng nghi

Case cụ thể – “self-healing pipeline”: nhiều trên blog tư vấn hơn postmortem kỹ thuật. Vòng lặp end-to-end (agent phát hiện drift → tự sinh dbt model → validate → ship) mới ở demo/roadmap – **chưa** case study production nào trích dẫn được.

Tầng đã ship (có primary source + ngày)

- **Databricks Lakeflow** – GA 2025-06-12: Connect, Declarative Pipelines, Jobs, dưới Unity Catalog (Databricks, 2025-06-12). ~15 connector, không “100+”; Designer **KHÔNG** GA.
- **Snowflake Openflow** – GA 5/2025, trên Apache NiFi, ban đầu chỉ AWS (Snowflake, 2025-05)
- Bằng chứng rõ nhất cho “agent cần semantic layer governed”: **Fivetran + dbt Labs sáp nhập** một công ty (2026-06-01)

Vendor lớn ship AI-assisted tooling để **soạn/giám sát** pipeline – vẫn human-in-the-loop cho thứ ghi ngược schema production.

Phản chứng đáng đặt cạnh: **91% pipeline dlt mới 1/2026 do agent viết** (số dlt tự báo, §3) – nhưng đó là **soạn thảo**, không phải **tự chữa lành**. Hai khái niệm khác nhau, dễ gộp nhầm.

- **Snowflake managed MCP server** – GA 2025-11-04 (trước đó preview), cùng GA cho **OAuth-based authentication** và **RBAC** trên MCP server/tool (Snowflake, 2025-11-04). Agent query được vào Snowflake account mà không cần tự triển khai hạ tầng riêng.
- **dbt MCP server** – open source từ 4/2025, chưa có nhãn GA rõ ràng (dbt Labs, 2025-04)
- Databricks MCP server, ClickHouse MCP server, OpenMetadata MCP tool support tồn tại như **ví dụ** – chỉ Snowflake được xác minh độc lập. “MCP đã thành chuẩn de facto” là superlative không nguồn; nói đúng hơn: một dịch chuyển 2025–2026 trong đó MCP trở thành giao diện được áp dụng rộng rãi cho data platform

Vì sao quan trọng cho agent của bạn

Ngày 8 bạn viết retrieval/tool-calling logic thủ công để agent chạm vào data. MCP server cho data platform là bước tiếp theo: agent tự query warehouse được governed qua một giao thức chuẩn, không cần bạn build custom integration cho từng data source.

Injected content trong DOCUMENT — hướng tấn công chạm vào pipeline data của bạn

- **GitHub MCP exploit, 2025-05-26:** một issue công khai dựng sẵn, yêu cầu agent đọc README mọi repo rồi liệt kê ra → agent tiết lộ tên private repo qua một PR nó tự tạo
- Phát hiện: Marco Milanta & Luca Beurer-Kellner (Invariant Labs)
- Chỉ tác dụng khi nội dung bị kéo vào context – KHÔNG lưu trong tool metadata

Lưu ý: Đừng dùng incident GitHub MCP làm bằng chứng cho độ-bền-trong-tool-metadata – nó dùng nội dung issue độc hại. Và: một định danh lỗ hổng kèm điểm nghiêm trọng đang lưu hành cho MCP tool poisoning **không truy được** về nguồn primary – con số “chính xác” không kèm methodology là dấu hiệu figure bịa.

Hướng tấn công thứ hai — poisoned tool **description** (Invariant Labs / whatsapp-mcp, Simon Willison 2025-04-09) – nằm trong metadata đăng ký của tool, không trong pipeline data của bạn.

Phòng thủ prompt injection, OWASP LLM Top 10, toàn bộ lớp bảo mật agent: **Ngày 11.**

Mốc	Kết quả
Ra mắt (2024)	GPT-4o: 86,6%→10,1% (Spider 1.0→2.0); o1-preview: 17,1%
2026, Spider 2.0-Snow	“Sentinel Agent v2 Pro” 96,70%; “Native mini” 96,53%
2026, 2.0-Lite (khó hơn)	trần ~76,23% (Tianqiong Data Agent + GLM 5.2)

Hàng 2024 đã nghỉ hưu – đừng dạy làm hiện trạng. Riêng con số Snow cũng phóng đại: hai split khác nhau, luôn nêu cả hai (Spider 2.0 leaderboard, 2026).

Leaderboard 2026 bị **agent harness nhỏ, tối ưu riêng cho benchmark** chiếm lĩnh (“Sentinel Agent”, “Native mini”, “Prism Swarm”, “ktx/Kaelio”) – submission thật, validated, **không phải** gian lận, nhưng tín hiệu **yếu** cho lựa chọn tool.

Với agent của bạn: đừng chọn model text-to-SQL production vì nó đứng đầu leaderboard hôm nay – tự benchmark trên domain data của chính bạn. Cùng bài học benchmark-gaming ở **Ngày 14**.

- **BIRD** (bird-bench.github.io): 95 database thật (33,4 GB), 12.751+ cặp câu hỏi-SQL, 37+ domain, cố ý gồm giá trị “bẩn”; thường SQL vừa đúng vừa **hiệu quả**
- **Human baseline** 92,96% (2025-12-16) vs. **Top system** AskData + GPT-4o, đội AT&T CDO-DSAIR: 81,95% (2025-09-25)
- Khoảng cách với con người: ~11–13 điểm, vẫn còn tính đến 2026

Lưu ý: Đừng lẫn đội AT&T giữa BIRD và Spider 2.0. Và đừng hard-code tên model đang dẫn đầu leaderboard trong slide – luôn trích thẳng URL, lấy giá trị hiện tại lúc tra cứu.

- **1,3 nghìn tỉ token**, lọc từ corpus Common Crawl **15 nghìn tỉ token** của FineWeb (khoảng 92% bị loại) (HuggingFace, FineWeb-Edu dataset card)
- Classifier chất lượng giáo dục huấn luyện trên **500.000 sample** được **Llama3-70B-Instruct** chấm thang 0–5
- HuggingFace phát hành **ablation model tái lập được** để cộng đồng tự kiểm chứng claim curation – thay vì tin lời
- **Nghịch trực giác (nguyên văn)**: *“We find that the deduplication of this dataset doesn’t have any impact on model performance in our ablation setup (1.8B trained on 350B tokens).”* – chống lại folklore “luôn luôn dedupe”

Changelog dùng DD-MM-YYYY (không phải MM-DD): v1.0.0 = 2 Jun 2024; v1.2.0 = 3 Jan 2025 (không phải 2025-03-01); v1.4.0 = 11 Jul 2025 (không phải 2025-11-07).

Với corpus RAG của bạn

FineWeb là case study công khai tốt nhất về “một pipeline mà công việc duy nhất là sản xuất corpus được chấm điểm chất lượng, dedupe, lọc” – một artifact **SỐNG**, có version, đo bằng chất lượng model/agent downstream, không phải row-count completeness. Corpus RAG Ngày 8 của bạn nên được đối xử giống vậy.

- Iceberg v3 / Delta 4.0 đã hội tụ phần lớn năng lực (§5) — điều đáng chú ý ở frontier là chuẩn hoá đã dịch lên tầng **catalog**: chuẩn thật là **Iceberg REST Catalog spec**, không phải vendor nào.
- **Diskless streaming (Kafka KIP-1150)**: ACCEPTED giữa tháng 1/2026 (không phải 2026-03-02 – đó là ngày sửa trang wiki lần cuối); đây là một **meta-KIP**, **CHƯA** ship trong Kafka stock tính đến 8/2026 – implementation thật nằm ở KIP-1163/1164

Lưu ý: “Giảm 80% TCO” là tuyên bố của chính **Aiven** (blog vendor, 2025-04-23), chưa qua audit độc lập – nếu dùng con số này ở đâu, phải gắn tên Aiven và ngày.

Nếu roadmap production cho RAG corpus của bạn dựa vào “Kafka diskless rẻ hơn 80%”, con số đó chưa audit và tính năng chưa ship – đừng lập budget dựa trên nó.

- **Meta Nimble + OpenZL**: OpenZL (nén format-aware) open-source khoảng 10/2025 (arXiv 2510.03203); 2026-07-05 Meta open-source luôn tích hợp Nimble↔OpenZL – tầng file-format của pipeline đang trở nên **AI-training-aware**
- **ByteDance StreamShield** (arXiv 2602.03189, nộp 2026-02-03, **preprint chưa xác nhận qua bình duyệt**): 70.000+ Flink job đồng thời, 11 triệu resource slot, recovery giảm từ **phút xuống giây**
- **Netflix Data Mesh + Streaming SQL** (2025): mở stream processing ngoài nhóm chuyên gia Flink, 20.000+ Flink job – nhưng **đừng** dựng “ngành hội tụ SQL-on-streams” từ đây: chỉ n=2 (Netflix + một talk Pinterest có nguồn duy nhất là một analyst vendor-adjacent)
- **Microsoft IQ** (Build 2026, blog Azure 2026-06-02): OneLake + lớp context định vị làm nền tảng **agent-grounding** – chỉ **Microsoft** có primary source; các cloud khác “có vẻ” đi cùng hướng, chưa phải fact đã kiểm chứng cho cả ba
- **Rename theater** (chỉ Google, 4/2026): BigLake → “Lakehouse for Apache Iceberg”; Dataplex Universal Catalog → “Knowledge Catalog” – API/IAM name **không đổi**. Bài học: rebrand không hàm ý re-architecture
- **BigQuery continuous queries**: chạy dưới user account tự dừng sau tối đa **2 ngày**, cần service account + orchestration để restart – không phải “chạy mãi, không cần ops” như marketing gợi ý (BigQuery docs, 2025–2026)

Đây là các lớp hạ tầng mà pipeline nuôi agent Ngày 8/9 của bạn có thể đứng trên trong 1–2 năm tới – biết để đọc vendor docs với con mắt phê phán, không phải để thi.

Lab 10, Checklist & Tổng Kết

Từ bài giảng sang tay bạn

17

- ☒ **Freshness SLI đã định nghĩa** trên artifact then chốt, SLO có **error budget** — không phải một threshold tĩnh (§9, §12)
- ☒ **Quality gate đặt TRƯỚC** khi ghi vào serving/Gold layer, và **fail-closed** — không fail-open (§7)
- ☒ **Idempotent re-run** — chạy lại job không tạo dữ liệu trùng lặp, không nhân đôi side-effect (§13)
- ☒ **Schema drift được phát hiện** — cột mới hoặc đổi kiểu âm thầm không lọt qua (§10)
- ☒ **Lineage được ghi lại** — map được từ source đến artifact, đủ để trace một cascade xóa hoặc một incident (§11)
- ☒ **Runbook đã viết** — đủ 6 bước từ detect đến post-mortem, rõ ai chịu trách nhiệm (§12)
- ☒ **Erasure path đã test** — xóa row ở nguồn KHÔNG đủ; đã kiểm tra xóa cả index/embedding phái sinh (§8)

Mục tiêu: Xây pipeline ETL nạp corpus cho chính agent RAG (Ngày 8) và multi-agent (Ngày 9) của bạn — ingest, clean, validate, chunk, embed, load vào vector store — rồi CỐ Ý corrupt một phần dữ liệu (xoá field, làm stale, đổi schema âm thầm) và đo lại câu trả lời của agent trước/sau.

Deliverable: Pipeline script chạy lại được mà không tạo trùng lặp (idempotent, §13); data quality report dùng **đúng** API GX 1.x — `gx.get_context() → context.data_sources.add_pandas() → batch.validate(expectation)`, **KHÔNG** phải `context.sources.pandas_default` tiền-1.0 (raise `AttributeError`) — cài trên Python 3.10–3.13 (GX 1.19.x); freshness + volume monitor (§9); bảng so sánh câu trả lời agent trên corpus sạch vs. đã corrupt.

Thời gian: 4 giờ (Vibe Coding 1.5 giờ + Lab 2.5 giờ)

Những ý chính cần nhớ trước khi sang bài tiếp theo

1

Silent vs. loud (§8–§10, §12): BI hỏng — dashboard vỡ. Agent hỏng — trả lời tự tin và sai, không exception nào được ném ra.

2

Quality gate đứng **TRƯỚC** serving layer, luôn **fail-closed** — nghi ngờ thì chặn, đừng để agent tự phát hiện data bẩn (§7).

3

Freshness cần SLO + error budget, không threshold tĩnh; schema drift + lineage biến “sai đâu đó” thành một điểm lỗi đo được — và idempotent re-run + runbook 6 bước biến nó thành incident có kỷ luật, không phải chạy lại job cho may rủi (§9–§13).

4

Xoá row Postgres **KHÔNG** xoá embedding — Ghost Vectors (arXiv 2606.18497, 2026-06): HNSW soft-delete để raw vector nguyên vẹn trên đĩa, bài toán **CHƯA** giải (§8).

Ngày 11 — Guardrails & AI Safety

“Bạn vừa học cách phát hiện khi pipeline âm thầm nạp data hỏng. Ngày mai: nếu data đó bị nạp hỏng KHÔNG PHẢI do lỗi ngẫu nhiên — mà do ai đó cố tình nhét vào?”

- Thêm input/output validation vào chính pipeline Ngày 10 bạn vừa build — schema check ở input, type/format check trước khi trả kết quả cho agent
- Viết 3 dòng: corpus RAG của bạn có thể bị **poison ngay tại ingest time** bằng cách nào? (Gợi ý: ai kiểm soát được nguồn dữ liệu bạn ingest, và pipeline của bạn có kiểm tra nội dung đó trước khi tin nó không?)

1. Sambasivan et al., *Everyone Wants to Do the Model Work, Not the Data Work*, ACM CHI 2021 (Google Research) — ~20 phút. Nguồn học thuật thay cho con số 80% đã gỡ (§15); phỏng vấn định tính n=53, không phải khảo sát population.
2. Sculley et al., *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 2015 — proceedings.neurips.cc — ~25 phút. Đọc vì CACE principle, pipeline jungles, glue code — KHÔNG vì một con số thời gian (xem §15).
3. Great Expectations, *Try GX Core* — docs.greatexpectations.io — ~15 phút. API 1.x đúng cho lab: `gx.get_context()...batch.validate()`; code kiểu `pandas_default` tiền-1.0 sẽ crash.
4. OpenLineage, *Getting Started & Object Model* — openlineage.io/docs — ~15 phút. Spec lineage đứng sau §11.
5. Quốc hội Việt Nam, *Luật Bảo vệ Dữ liệu Cá nhân số 91/2025/QH15* (PDPL, hiệu lực 2026-01-01) + Nghị định 356/2025/NĐ-CP — ~10 phút (tóm tắt: dlapiperdataprotection.com). Ràng buộc thật lên kiến trúc RAG nếu pipeline đẩy data ra nước ngoài.
6. Chakrabortii et al., *Ghost Vectors: Soft-Deleted Embeddings Remain Reconstructible in HNSW Vector Databases*, arXiv 2606.18497 (2026-06-16) — ~10 phút (abstract + kết luận). Vì sao xóa row Postgres không đủ (§8).

Hỏi & Đáp

Agent của bạn có thể tự phát hiện rằng nó đang trả lời sai không — hay luôn cần một hệ thống bên ngoài, tách biệt, phát hiện giúp nó?



Cảm ơn!

AICB Phase-1 · Ngày 10

Data Pipeline & Data Observability

Tiếp theo: Ngày 11 — Guardrails & AI Safety

github.com/vbi-academy/aicb-phase1

Phụ Lục A — Cầu Nối Sang Ngày 13

Quan sát agent, không chỉ quan sát data

18

- Một **trace** = một lượt chạy agent **end-to-end**, từ câu hỏi người dùng gõ vào đến câu trả lời cuối cùng hiện ra; với agent RAG (Ngày 8) hay multi-agent (Ngày 9), một trace có thể đi qua nhiều bước: retrieval, tool call, handoff giữa các agent
- Một **span** = một **bước bên trong** trace đó: một lần retrieval, một lần gọi tool, một lần LLM sinh text
- Nhiều span ghép lại theo thứ tự thời gian → dựng lại toàn bộ trace
- Không đo được span nào chậm/sai → không biết bước nào trong agent đang gây vấn đề
- Trace là đơn vị bạn **debug theo**, không phải log dòng-đơn lẻ

Phía nội dung (input/output)

- **Prompt** gửi cho LLM ở từng bước
- **Chunk được retrieve** – đúng những đoạn corpus nào được đưa vào context
- **Tool call** – tên tool, tham số, kết quả trả về

Phía vận hành (chi phí/hiệu năng)

- **Token count** – input/output token mỗi bước
- **Latency** – mỗi span mất bao lâu
- **Cost** – quy đổi token thành tiền

Vì sao quan trọng

Đây chính là dữ liệu bạn cần để trả lời “bước nào trong agent của mình đang chậm, đắt, hoặc sai” – **chi tiết công cụ và cách dựng dashboard ở Ngày 13.**

Faithfulness của câu trả lời

- So khớp câu trả lời của agent với đúng những chunk đã retrieve trong trace đó
- Câu trả lời có **bịa thêm** thông tin không có trong context không?
- Đây là cách bắt **silent failure** (§9) ngay tại điểm agent trả lời, chứ không chỉ ở điểm ingest dữ liệu

Sức khỏe vector store (nổi §9)

Sức khỏe vector store: **cùng 3 metric như §9** (embedding coverage, index version consistency, retrieval staleness).

Preview

§9 đo corpus *tĩnh*. Ngày 13 đo corpus đó *tại đúng thời điểm* một câu trả lời cụ thể được tạo ra – chi tiết cách đo ở Ngày 13.

Giai đoạn	Tín hiệu §9 (data)	Tín hiệu agent (Ngày 13)
Ingest / source row	Freshness, Volume, Schema	–
Corpus / vector store	Embedding coverage, Contradiction rate	Index version consistency
Retrieval (mỗi query)	–	Chunk retrieve được ghi vào span
Tool call / reasoning	–	Span: tool, tham số, kết quả
Câu trả lời hiện ra	–	Faithfulness, token, latency, cost

Một pipeline, một vòng lặp

§9 quan sát nửa đầu (dữ liệu trước khi vào agent). Ngày 13 quan sát nửa sau (agent xử lý dữ liệu đó ra sao). Thiếu một nửa, bạn chỉ biết “pipeline chạy đúng” hoặc “agent chạy đúng” – không bao giờ biết **vì sao** một câu trả lời cụ thể sai.